以尾數位元換瓦數：在 GH200 功耗上限下以尾數歸零提升科學計算效能

專題報告 · @Koying Chen · Oct 5, 2026

摘要

現代超級電腦與 GPU 叢集越來越常在功耗上限下運作：機房依機櫃電力限電，學生超算競賽限制整個叢集的功率，NVIDIA GH200 也讓使用者直接設定整顆模組的功耗上限。在功耗上限下，GPU 會把時脈降到剛好不超過上限，所以同一個程式跑多快，取決於它每次運算消耗多少電；而 GPU 的耗電又隨資料而變，位元翻轉越多越耗電。另一方面，許多科學應用實際需要的精度，比 FP64 提供的 52 個尾數位元少得多。

本專題研究「尾數歸零」：以軟體把運算元低位的尾數位元設為 0，不改變資料格式與演算法，讓運算單元與資料線路少翻轉、少耗電，再把省下的電在功耗上限下換成更高的時脈與吞吐量。我們在 14 節點的 GH200 叢集上，以成對交錯的量測與「等效功耗」（原生版本要多給幾瓦才追得上）評估這個做法，涵蓋矩陣乘法微基準、8 節點 HPL 與 HPL-MxP、Quantum ESPRESSO 與 PySCF 等科學軟體、第三方非矩陣乘法程式，以及 AI 訓練與推論。

主要結果：在功耗受限、截斷直接做在程式內的情況下，以矩陣乘法為主、有正確性檢查的完整程式都得到正面收益。8 節點 HPL 保留 36 位（rocHPL 含隱藏位的有效位數，即 35 個尾數位元；殘差離門檻仍有約百倍餘裕）在 400–800 W 快 4.0–11.0%，相當於多給 39–127 W 的功耗上限（調參後原生在 1000 W 為 394 TFLOP/s）；HPL-MxP 保留 3 位元快 1.8–12.7%（對角優勢的測試矩陣）；Quantum ESPRESSO 依求解器的收斂門檻自動調整位元數，在總能量印出的每一位都不變的情況下快 2.4–4.2%、少耗電 2.5–3.9%（金表面的收益約一半來自 Davidson 迭代變少）；在高耗電的 DF-MP2 上，截斷 20–32 位在 500–700 W 快 4.9–8.3%、1000 W 時省電 8–14%；在跨度寬又吃滿電的高斯過程地統計上，截斷 28 位在 500–700 W 快 7.4–8.2%（只估範圍參數的最大概似估計快 7.1–7.3%，估出的參數與原生相同），而 FP32 結果不可用、INT8 模擬較慢。機制實驗顯示，62–97% 的省電來自低位元「不再翻轉」，在 tensor core、FP64 乘加單元與記憶體搬移上都成立。我們也整理出它的適用範圍：與 INT8 模擬 FP64 的分界點隨矩陣形狀與功耗上限而變；把所有 GEMM 換成 FP32 或 3xTF32，在長內積的 Quantum ESPRESSO 上無法收斂；列內指數跨度寬時，固定位數的 INT8 模擬可能安靜地算錯：在高斯基底 DFT 上，cuBLAS 固定 36 位的模擬錯了 52 mEh（加到 40 位或改用 cuBLAS 的自動模式才正確，但時間分別是截斷的 1.5 倍與原生的 2.1 倍），而同樣保留 36 位的截斷（最近捨入）只差 1e-6 Eh；不過跨度寬不一定出錯（高斯過程上的模擬只是較不準），以 tile 為單位換格式的混合精度也能用在這類資料上，截斷還能疊加在它之上；在已使用 FP8 的 AI 推論上，FP8 已拿走幾

乎全部的好處。尾數歸零每個元素只多一次位元運算、不需要新硬體，是功耗受限系統上容易採用的調整手段（寫進函式庫才能免去攔截時的複製成本），最適合有殘差修正或自洽迭代的科學求解器。

（本報告中所有截斷後的 HPL／HPL-MxP 與 MLPerf 數字都是研究用途，不是 TOP500 或 MLPerf 的正式成績。）

1 研究動機與問題

1.1 功耗成為算力的上限

大型計算系統的電力與散熱都有預算。機房依機櫃的供電能力設定功耗上限，學生超算競賽（SCC）限制整個隊伍的總功率，NVIDIA GH200 superchip 也讓使用者直接設定整顆模組（GPU、CPU 與記憶體）的功耗上限，範圍是 200–1000 W。當程式的耗電碰到上限時，GPU 的功耗管理會降低時脈與電壓，直到功耗不超過上限。這代表在功耗受限的情況下，效能不再取決於硬體最高能跑多快，而是取決於「每次運算花多少電」：只要讓每次運算少耗一點電，時脈就能拉高一點。

1.2 功耗隨資料而變

CMOS 電路的動態功耗約為 $\alpha \cdot C \cdot V^2 \cdot f$，其中 α 是切換活動率：資料位元在相鄰時脈之間翻轉（0 變 1 或 1 變 0）得越多，電路充放電的次數越多，耗電就越高。所以同一個程式、同樣的運算量，輸入資料不同，耗電就不同。前人已在 A100 與 GH200 上量到：同樣的矩陣乘法，隨機輸入比固定值或全零輸入多耗 60–67% 的功率（第 3 章）。

1.3 應用用不到全部的尾數

FP64 有 52 個尾數位元，約等於 16 位十進位有效數字。但很多應用並不需要這麼多：HPL 只要求殘差小於門檻；平面波密度泛函計算的自洽迭代在前期只需要很鬆的精度；AI 推論甚至用 FP8（只有 3 個尾數位元）也能維持準確度。這些用不到的低位元在運算中持續隨機翻轉，就是白白消耗的電。

1.4 研究問題

把這三點合起來，就得到本專題的核心想法：**用軟體把用不到的低位尾數設為 0，讓它們不再翻轉，再把省下的電在功耗上限下換成效能。** 具體要回答三個問題：

1. **機制：** 為什麼少幾個尾數位元就會省電？是因為位元不翻轉，還是因為數值是 0？
2. **效益：** 在功耗上限下，真實的科學應用能換到多少效能與省電？要付出多少精度？

3. **適用範圍：** 和「直接換成較小的格式」（INT8 模擬 FP64、FP32／3xTF32、FP8）相比，它在哪裡有價值？

1.5 報告架構

第 2 章介紹浮點格式、GPU 功耗與功耗上限的背景知識；第 3 章整理相關研究；第 4 章說明研究方法與實驗平台；第 5 章是實驗結果（包括高耗電的 DF-MP2、指數跨度實驗，以及跨度寬又吃滿電的高斯過程與 ExaGeoStat）；第 6 章討論適用範圍與限制；第 7 章是結論與未來工作。

2 背景知識

2.1 浮點格式與尾數

一個浮點數由符號、指數與尾數（fraction）三部分組成，數值為 (−1)^s × 1.f × 2^(e − bias)。指數決定「值域」（能表示多大多小的數），尾數決定「精度」（有幾位有效數字）。本報告用到的格式：

格式	符號／指數／尾數位元	約略的十進位有效位數	常見用途
FP64	1／11／52	16	科學計算、HPL
FP32	1／8／23	7	一般單精度
TF32	1／8／10	3	tensor core 的 FP32 加速模式
BF16	1／8／7	2–3	AI 訓練與推論
FP8 E4M3	1／4／3	1	AI 推論與訓練

本報告說「保留 k 位」，指的是保留最高的 k 個尾數位元、把其餘設為 0。例如 FP64 保留 36 位，就是把 52 個尾數位元中最低的 16 個清為 0。符號與指數不動，所以值域不變，只是每個值變得比較「粗」：直接截斷時，每個值的相對誤差不超過 2^{-k}；最近捨入時不超過 $2^{-(k+1)}$。格式與程式完全不用改，這是它和「換成較小格式」最大的不同。

2.2 GPU 功耗與位元翻轉

GPU 的動態功耗約為 $\alpha \cdot C \cdot V^2 \cdot f$。資料在暫存器、運算元匯流排、乘法器與記憶體之間流動時，只要某個位元和前一筆資料不同，這條線就要充電或放電。隨機資料的低位尾數幾乎每次都在翻轉；如果把它們全部設為 0，相鄰兩筆資料在這些位置就永遠相同，這些線路就不再切換。在乘

法器裡，一個運算元的低位是 0，對應的部分積也恆為 0，整片電路都不動。記憶體方面，HBM 與 DRAM 的存取能耗也隨資料的翻轉率變化。

2.3 功耗上限與時脈

GH200 的模組功耗上限涵蓋 GPU、Grace CPU 與記憶體。GPU 會持續調整時脈，讓功耗不超過上限（實測撞上限時的 1 秒平均功耗常低於上限，限電器看的應是更短時間尺度的功耗）。所以會出現兩種情況：

- **功耗受限**（例如上限 500 W）：每次運算少耗電，GPU 就能在同樣的功耗下跑更高的時脈，表現為**變快**。
- **不受限**（上限高於程式的用電，時脈停在最高的 1980 MHz）：速度不變，表現為**少耗電**。

1000 W 是 GH200 的預設也是最高上限，但吃滿電的程式（例如 FP64 矩陣乘法與 HPL）在 1000 W 仍有部分時間撞上限，所以兩種效果會同時出現。

2.4 其他「降精度換速度」的做法

- **INT8 模擬 FP64（Ozaki 方法）：** 把 FP64 數拆成多片低精度整數，用 INT8 tensor core 分別相乘再合起來。片數越多越準。cuBLAS 13 內建這種模擬；GEMMul8 是第二版 Ozaki 方法，用中國餘數定理把數值拆到多個模數。
- **FP32 與 3xTF32：** 把 FP64 改用單精度計算。3xTF32 把每個 FP32 數拆成兩個 TF32，以三次 tensor core 乘法達到接近 FP32 的精度。
- **FP8：** Hopper 上 AI 推論的主流格式，資料寬度比 BF16 少一半，矩陣乘法的峰值是 BF16 的兩倍，靠每個張量或每個通道的縮放因子彌補較窄的值域。
- **混合精度迭代修正：** 用低精度做 LU 分解，再用 FP64 迭代修正到高精度，HPL-MxP 就是這樣的基準。

這些做法都「換格式」，靠吃吞吐量更高的運算單元加速；尾數歸零則沿用原本的運算單元，只降低輸入的翻轉。兩者有各自的適用場合，這是第 5.5 節要回答的問題。

2.5 本報告使用的基準與應用

- **HPL：** 解稠密線性系統的 FP64 基準，TOP500 排名的依據。最後會檢查縮放後的殘差是否小於 16，超過就判定失敗。
- **HPL-MxP：** 用 BF16 做 LU、再用 FP64 迭代修正的混合精度基準。
- **Quantum ESPRESSO：** 平面波密度泛函理論軟體，以自洽場（SCF）迭代求基態，每圈用 Davidson 對角化，收斂門檻會逐圈收緊。
- **PySCF（GPU4PySCF）：** 高斯基底的量子化學軟體。

- **MFEM、SW4、LAMMPS：** 不以矩陣乘法為主的 FP64 程式（有限元素、地震波、分子動力學）。
- **MLPerf Llama 2 70B LoRA 與六個語言模型：** AI 訓練與推論。

3 相關研究

3.1 資料相依的 GPU 功耗

Bhalachandra 等人在 A100 上發現，同樣大小的 DGEMM，隨機運算元比固定值多耗 67% 功率 [Bhalachandra2022]。Gregersen 等人在 A100、H100、V100 上改變 GEMM 輸入的分佈與位元樣式，功耗最多差近 40%，其中把最低位元歸零可降低功耗；但他們刻意選擇不會降頻的矩陣大小，沒有量功耗上限下的效能 [Gregersen2024]。He 在 A100 上示範全零輸入的矩陣乘法比隨機輸入快約 15%，並把功耗上限從 330 W 逐步降到 100 W：可預測輸入的加速隨上限降低而變大（圖中約從 1.2 倍增到 150 W 的 2.3 倍），到 100 W 才轉而變小 [He2024]。Antepara 等人在 GH200 上量到零輸入的 DGEMM 只用 563 W、隨機輸入 898 W [Antepara2025]。Lucas 與 Juurlink 建立的 GPU ALU 功耗模型中，主項是位元翻轉數 [Lucas2016]。這些工作都用全零、常數等特殊輸入示範現象，沒有截斷真實資料的部分尾數，也沒有評估數值精度或真實應用。

功耗受限的 HPC。 Rountree 等人最早在硬體強制的功耗上限下量測效能，發現同型處理器在同一上限下的執行時間會因製程差異而不同（約 6%）[Rountree2012]；Inadomi 等人在大型系統上量到，同一功耗上限下各節點的 DGEMM 效能可差到 64%，並依變異分配功耗 [Inadomi2015]。Patki 等人在固定的總功耗下多裝節點、再以上限分配功耗（超量配置）[Patki2013]；Zeus 在 GPU 上依功耗上限與批次大小調整 DNN 訓練的能耗 [Zeus2023]。這些工作調的是「功耗怎麼分配」，本專題則降低每次運算所需的電，兩者可以疊加；我們量到同一上限下節點間 13–17% 的時脈差異（第 4.4 節），也與它們的觀察一致。

3.2 硬體層的尾數遮罩

「把尾數設為 0 來省電」在硬體設計上早有前例：Tong 等人的可變位寬浮點乘法器每次運算的能量最多少 66% [Tong2000]；Pool 等人為行動 GPU 的著色器選擇精度，以能耗模型估計頂點轉換可省 23–36% 的能量 [Pool2008]，像素著色器的算術最多可省 71% [Pool2011]；Guenther 等人在基頻處理器的運算單元輸入端遮罩尾數，最多省能 23% [Guenther2016]；AMD 的專利在 GPU shader 中加入硬體歸零模組 [AMD2022]。它們都需要新硬體或以模擬器估算。資料搬移方面，DRAM 與 HBM 的能耗隨資料的翻轉率變化 [Chatterjee2017, OConnor2017, Ghose2018]。

3.3 精度調校與 FP64 模擬

Precimonious 等工具自動把變數換成較低精度的型別，目標是速度 [RubioGonzalez2013]；rpe 與 chop 用同樣的位元操作模擬低精度，但不量功耗 [Dawson2017, Higham2019]。transprecision computing 主張依需要在時間與空間上細粒度地調整精度 [Malossi2018]；Klöwer 等人從資料本身估計真正承載資訊的尾數位數，再以最近捨入去掉其餘位元來壓縮氣候資料 [Klower2021]。Ozaki 方法 [Ozaki2012] 在整數矩陣單元上模擬 FP64 [Ootomo2024, Ozaki2025]，Uchino 等人評估了它的效能與功耗 [Uchino2025]。cuBLAS 內建的定點模擬對 A 的每一列共用一個縮放因子，列內的動態範圍要由尾數位元承擔；它的自動模式依指數跨度決定切片數，所需位元超過上限時改回原生 FP64 [cuBLASEmu, Schwarz2026]。形式上最接近本專題的是 Grützmacher 等人：把 FP64 值依尾數切段存放、保留完整指數，只讀前面的段落，讓受頻寬限制的 PageRank 快 10–30% [Grutzmacher2020]；他們截斷是為了省頻寬，本專題是為了降低運算的切換功耗。

地統計的混合精度。 ExaGeoStat 以 FP64 的 tile Cholesky 計算高斯過程的概似 [Abdulah2018]。之後的混合精度版本先把位置依空間填充曲線排序、讓強相關集中在對角線附近，對角 tile 留在 FP64，再依離對角線的距離或 tile 的範數把整個 tile 轉成 FP32、FP16 或 FP8：FP64／FP32 平均快 1.6 倍 [Abdulah2019]，加上 FP16 最多 2.64 倍 [Abdulah2022]，結合低秩近似最多 12 倍 [Cao2022]，在 GH200 上以四種精度比 FP64 快 3 倍 [Ren2025]，也用在 GH200 系統上的氣候模擬器 [Abdulah2024]。Cao 等人在單張 V100、A100、H100 上量過這類分解的功耗與能耗 [Cao2023]，但沒有設功耗上限、也沒有分析時脈。單一 FP32 不足以計算這類概似 [Abdulah2022]，與第 5.10 節的結果一致。這些做法都是把整個 tile 換成別的格式；本專題保留 FP64、只截斷運算元的尾數，兩者可以疊加。GPU 上的 RI-MP2 也曾用單精度與混合精度 GEMM 加速 [OlivaresAmaya2010]。

3.4 依收斂程度調整精度

Woo 等人在 LOBPCG 特徵值求解器中依收斂狀態在 FP32、FP64 與混合精度之間切換 [Woo2023]；Huang 等人在 GPU4PySCF 中依 SCF 能量變化選擇 INT8 模擬的切片數 [Huang2026]；Dawson 等人發現量子化學的密度矩陣純化約 37 位有效尾數就足夠 [Dawson2024]。它們都靠換到吞吐量較高的格式加速，沒有研究功耗。

3.5 本研究的定位

工作	需要新硬體	截斷真實資料	功耗上限下量效能	驗證應用的數值結果
Bhalachandra、Gregersen、Antepara	否	否（全零或特殊樣式）	否	否

工作	需要新硬體	截斷真實資料	功耗上限下量效能	驗證應用的數值結果
He	否	否	是（330→100 W 掃描，全零輸入）	否
Tong、Pool、Guenther、AMD	是	是	否	部分
Woo、Huang	否	換格式	否	是
Grützmacher（尾數切段儲存）	否	是（留指數、丟尾數，為了省頻寬）	否	是（PageRank）
ExaGeoStat 系列（Abdulah、Cao、Ren）	否	換格式（整個 tile）	否（Cao2023 量過功耗，未設上限）	是
本研究	**否**	**是**	**是（含 8 節點 HPL）**	**是**

據我們所知，本研究是第一個在現役 GPU（GH200）上以純軟體截斷 FP64／BF16 運算元尾數來降低切換功耗、把省下的電在功耗上限下換成吞吐量，並在真實 HPC 應用上以物理誤差驗證的系統性研究。在 FP64 內「留指數、丟尾數」本身並不新 [Grutzmacher2020]，地統計的混合精度也已有人量過功耗 [Cao2023]，全零輸入在不同功耗上限下的加速也已有人量過 [He2024]；本專題的不同在於目的（降低切換功耗）、做法（不換格式）與量測條件（功耗上限下的時脈與吞吐量）。

4 研究方法

4.1 尾數歸零與三種捨入方式

對一個有 p 個尾數位元的運算元（FP64 為 52、BF16 為 7、FP8 E4M3 為 3），保留最高的 k 個位元，其餘 p − k 個設為 0。比較三種去掉低位的方式：

- **截斷**：直接用一個位元遮罩清除低位（往 0 的方向）。
- **最近捨入**：先加上保留的最低位權重的一半（也就是被捨去部分的最高位）再遮罩。攔截工具、GEMM 微基準、Quantum ESPRESSO、DF-MP2 與高斯過程的實作在平手時往離零較遠的方向捨入；HPL、LoRA 的權重載入與語言模型推論的實作則是平手取偶。
- **隨機捨入**：先加上一個小於被捨去位元權重的隨機整數再遮罩。

每個元素只需一個整數 AND（捨入再加一個加法），可以融入產生該運算元的核心；本報告的 HPL 另跑一個捨入核心、攔截工具另做一次複製，這些額外成本都算在量到的時間裡。本報告說「截斷到 k 位」時泛指保留 k 位；除非特別註明，應用程式（Quantum ESPRESSO、PySCF、DF-MP2、高斯過程、ExaGeoStat、第三方程式）與 HPL 用最近捨入，GEMM 微基準、分界點掃描與 HPL-MxP 用往零截斷。HPL 的位數依 rocHPL 的慣例是含隱藏位的有效位數：36 位相當於保留 35 個尾數位元。

4.2 四種實作層次

1. **在基準程式中歸零：** 矩陣乘法微基準在產生輸入時歸零；HPL 在每次 FP64 GEMM 前以一個捨入核心處理兩個輸入（執行時以環境變數指定位數）；HPL-MxP 在編譯時指定位數，每次寫回 BF16 時歸零。
2. **攔截函式庫：** 對不修改原始碼的科學軟體，以 LD_PRELOAD 攔下 cuBLAS 的所有 FP64／複數 FP64 矩陣乘法入口，把 A、B 複製並捨入後再呼叫原函式（呼叫端的資料不變）。同一個函式庫也能統計運算元的指數分佈、切換成 cuBLAS 的 FP64 模擬，或改用 FP32／3xTF32 計算，用於公平比較。
3. **程式內改寫：** 在 Quantum ESPRESSO 的 Davidson 對角化裡，對波函數區塊原地歸零，不需要複製；第三方非矩陣乘法程式（MFEM、SW4、LAMMPS）則在設定階段對儲存的係數陣列歸零一次；GPU4PySCF 的 DF-MP2 在三指標張量建好後原地歸零一次，高斯過程核心則在每個面板做尾端更新前原地歸零。
4. **依收斂門檻自動調整：** Davidson 每圈會收到一個收斂門檻 ethr，整個 SCF 過程中從約 10^{-2} 收緊到 10^{-12}。我們保留 $k = \mathrm{clamp}(\lceil -\log_2 \mathrm{ethr} \rceil + m, 16, 52)$ 個位元（m 是餘裕），讓前期用少位元、最後幾圈才用足。這是一條經驗規則：ethr 是特徵值的絕對容許誤差（單位 Ry），與尾數的相對誤差不同，m 用來吸收兩者的差距；它不需要對應用建模，只用程式本來就會計算的容許誤差。

LoRA 訓練則在載入後對凍結的 FP8 基底權重歸零一次；推論實驗在線性層中模擬各種格式，FP8 使用真實的 FP8 tensor core 矩陣乘法。

4.3 實驗平台

14 個節點，每個節點一顆 NVIDIA GH200 480GB superchip（72 核 Grace CPU、Hopper GPU 搭配 96 GB HBM3、最高 SM 時脈 1980 MHz），以 200 Gb/s InfiniBand 連接，由 Slurm 管理。模組功耗上限可設 200–1000 W（預設 1000 W；規格表列出的可設範圍是 450–1000 W [NVIDIAGH200]，本報告低於 450 W 的設定超出規格表，但驅動程式接受，實測功耗也貼著上限）。軟體為 NVHPC 26.5（CUDA 13.2、cuBLAS 13、NCCL、HPC-X Open MPI）。多節點實驗（HPL、HPL-MxP、LoRA）用 8 個節點。

4.4 量測與統計

- **功耗：** 每 250–500 ms 以 nvidia-smi 取樣模組功耗、SM 時脈、溫度與降頻原因。耗電量以驅動程式的 1 秒平均模組功耗對時間積分（對忽高忽低的負載，單一瞬時讀數會偏差 −37% 到 +41%）；表中的平均功耗（W）是視窗內瞬時取樣的平均，穩態下與 1 秒平均相差約 1% 以內。
- **公平比較：** 每次量測都等 GPU 降到 45 °C 以下才開始；同一功耗上限內以交錯順序（A、B、B、A）在同一組節點上執行各變體，消掉溫度與先後順序的影響。
- **信賴區間：** 同一節點、同一上限、同一輪的「歸零 ÷ 原生 − 1」為一個成對樣本，以 t 分布取 95% 區間，這樣可消掉節點間 13–17% 的時脈差異。區間只用在矩陣乘法微基準（3 個節點各 2 對）與 Quantum ESPRESSO 的自動調整（6 對）；同一節點的兩對並非完全獨立，區間可能偏窄。HPL、HPL-MxP、DF-MP2 與高斯過程每格只有同一組節點上的兩輪，表中列兩輪的值或平均，不附區間。每個工作的第一次執行常偏慢（暖機），而交錯順序的第一個多半是原生，會讓第一對的收益偏大（第 5.4 節）。
- **資料檢查：** 每個量測視窗都記錄實際生效的功耗上限，與預定不符的一律剔除（叢集是共用資源）；硬體功耗煞車以降頻計數器判讀（只在診斷工作 1675 開啟，主要的 HPL／HPL-MxP 掃描沒有記錄）。

4.5 等效功耗 ΔW

在功耗上限 P 下，歸零版本的效能記為 $p'(P)$。沿著原生版本實測的「效能對上限」曲線線性內插，找出原生版本達到 $p'(P)$ 所需的上限 P_{native}，定義 $\Delta W = P_{native} - P$。它的意思是：未修改的程式要追上歸零版本，每個節點要多給幾瓦。這比百分比更直觀地對應到機房的電力預算。上限之間以直線內插；效能對上限的曲線向下彎，所以兩個上限相隔較遠時（例如 800–1000 W），ΔW 會略為高估。

4.6 誤差模型

兩個輸入都保留 k 位時，矩陣乘法結果的相對誤差（Frobenius 範數）約為 $\alpha \cdot 2^{-k}$，與內積長度無關（累加仍是 FP64）。均勻隨機資料上（$k \geq 4$）最近捨入 $\alpha \approx 0.27$、隨機捨入 0.38、截斷 0.70，截斷的誤差約是最近捨入的 2.6 倍。截斷的誤差幾乎全是系統性的縮小，對會累積偏差的流程有害，這是科學軟體應該用最近捨入的原因（第 5.4 節）。

5 實驗結果

本章依「機制 → 微基準 → 完整基準 → 科學軟體 → 與其他做法比較 → 非矩陣乘法程式 → AI」的順序呈現。除非特別說明，百分比都是和同一功耗上限、同一節點上的原生版本比。

5.1 省電機制：不翻轉還是數值為零？

尾數歸零同時改變兩件事：低位元不再隨資料翻轉，而且數值變成 0。為了分開兩者，我們把被捨去的位元改填全 1，或一組所有元素共用的固定非零樣式：這兩種填法也不會翻轉，但數值不是 0。在 1000 W 對三個層次各量一次（填值版本都跑在最高時脈 1980 MHz，比的是功耗；原生 FP64 矩陣乘法仍有 95% 的時間撞上限、只有 1822 MHz，它的功耗是被上限截住的值，所以「不翻轉」所佔的比例算得偏保守）：FP64 tensor core 矩陣乘法、只用一般 FP64 運算單元的乘加迴圈，以及純記憶體搬移。

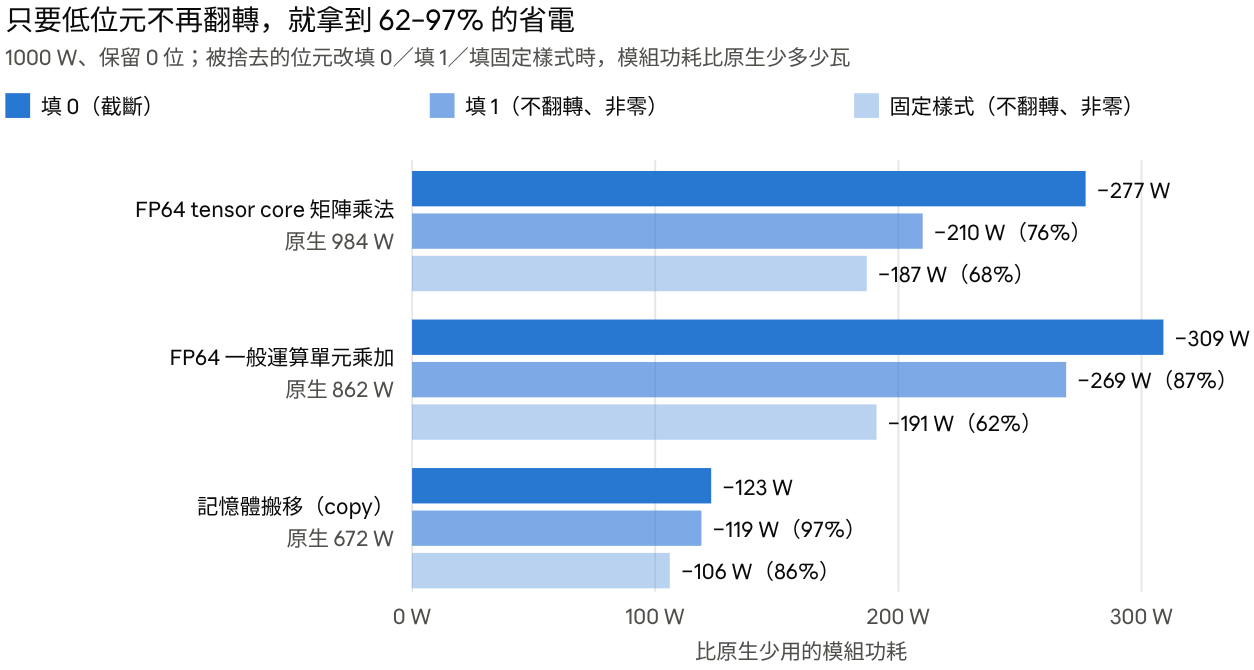


圖 5.1　Slurm 工作 1752–1753、1762、1815、1858；gh200/results/m1_toggle_summary.csv 等

三個層次的結論一致：只要讓低位元不再翻轉，就拿到完整省電的 62–97%；填 0 比填 1 或固定樣式再多省一截。記憶體搬移幾乎完全是「不翻轉」的效應（86–97%）；乘法器則還有額外的「數值為 0」效應：保留 20 位時，填 1 的低位與另一個運算元相乘的部分積仍會隨它翻轉，填 0 則整片不動；保留 0 位時兩個運算元的尾數都是常數，部分積本來就不翻轉，填 0 仍比填 1 省（707 對 774 W），可能來自乘積的對齊與累加較單純（全 0 時乘積是 2 的冪），尚未驗證。在 500 W（功耗受限、比較效能；單一節點、各兩次）重做同樣的對照，「不翻轉」佔效益的 77–92%，結論相同。這在 GH200 的模組功耗上量化了前人提出的翻轉假說 [Gregersen2024]；資料搬移的部分與 DRAM 能耗隨翻轉率變化的觀察一致 [Chatterjee2017, Ghose2018]。

5.2 功耗上限下的矩陣乘法微基準

先在最單純的工作負載上量出效益的上界。測試用 cuBLAS 矩陣乘法（FP64 32768×32768×2048、BF16 32768×32768×6144），輸入是 [−1, 1) 的均勻隨機值。3 個節點、300–1000 W 八個上限，每個上限以交錯順序各跑 2 次。

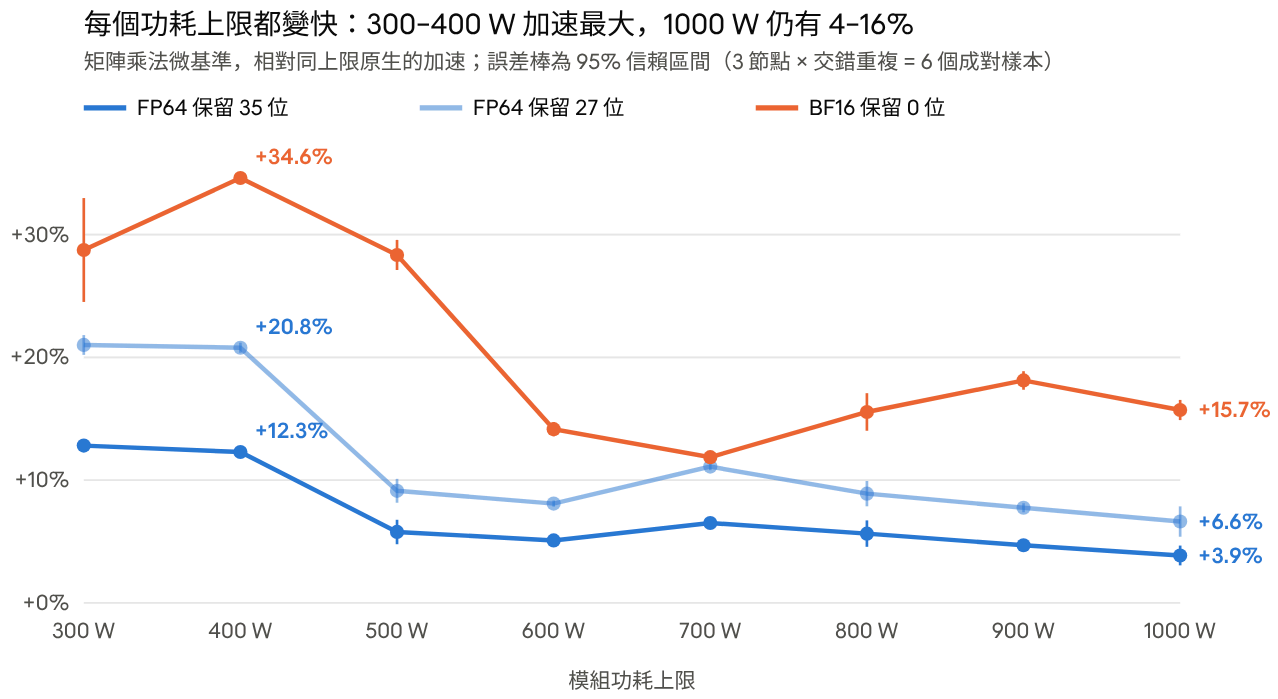


圖 5.2　Slurm 工作 1667–1669（work1–3）；gh200/results/e1_ci.csv

大多數信賴區間只有 ±0.1–1.2 個百分點，效益遠大於量測雜訊。換算成等效功耗，FP64 保留 27 位在 500–800 W 相當於多給 79–162 W，BF16 保留 0 位相當於多給 121 W 到 200 W 以上。最佳能效也提高：FP64 從原生的 82 GFLOP/J（500 W）升到保留 27 位的 90 GFLOP/J（400 W），BF16 從 950（600 W）升到保留 0 位的 1159 GFLOP/J（500 W）。效益與功耗上限不是單調關係，300–400 W 最大。一個可能的解釋（尚未驗證）是：上限極低時電壓已在下限，功耗大致與時脈成正比，省下的電能換到較多時脈；而 1000 W 時 FP64 與 BF16 矩陣乘法仍踩在功耗上限，所以仍有 3.9%（FP64 保留 35 位）到 15.7%（BF16 保留 0 位）的加速。

需要強調的是：這是均勻隨機資料、保留位數很低時的數字，是翻轉最劇烈的情況，是效益的上界，不是應用的操作點。下一節看有正確性檢查的完整程式。

5.3 HPL 與 HPL-MxP：有正確性檢查的完整基準

HPL。 用 HPL 內建的量化選項，在每次 FP64 矩陣乘法前以一個捨入核心把兩個輸入捨入到 k 位（平手取偶的最近捨入），乘法本身仍是 FP64 cuBLAS。8 節點、N = 286720（調參後的設定，見下）、400–1000 W，每個上限以交錯順序各跑 2 次。

先把原生 HPL 調到最好。 最早的量測沿用 H200 時期的設定（N = 262144、NB = 1024、2×4、f = 0.05、NCCL 預設），1000 W 原生只有 374.8–376.0 TFLOP/s。我們在 1000 W 掃描 N、NB、P×Q、f 與 NCCL 參數（Slurm 工作 3303、3306，24 組設定、26 次執行）：NCCL 固定 8 個通道、2 MiB 緩衝貢獻約 4.5%，N 加大到 286720 再多約 2%，4×2 網格反而慢

14%。同一個最佳設定重跑三次是 390.2–398.8 TFLOP/s（差約 2%），f、NCCL 通道與緩衝、NB、N 的其他鄰近設定都沒有超出這個雜訊（f = 0.15 反而慢 3%）。下面的上限掃描都用這個設定（Slurm 工作 3315，work4–11）。

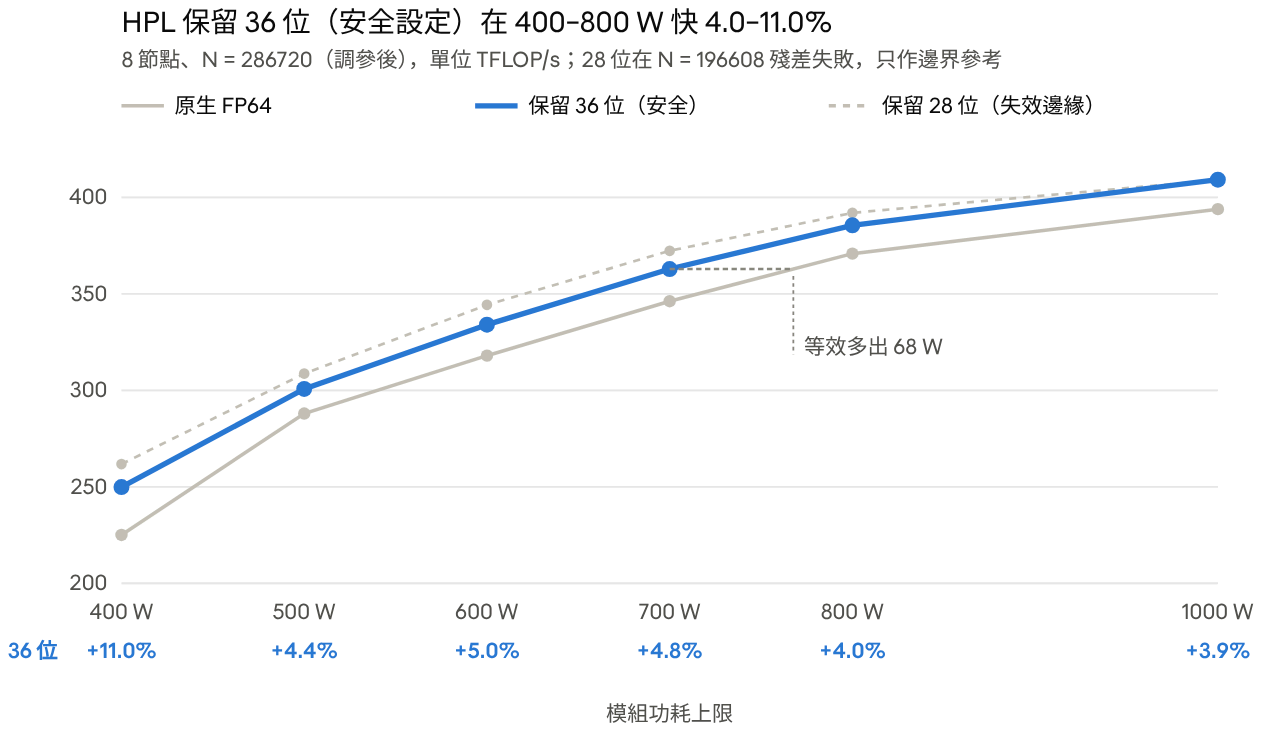


圖 5.3　Slurm 工作 3315（調參後的功耗上限掃描）、1820（多組 N）；gh200/results/e3_tuned_summary.csv

保留 36 位在 400–800 W 快 4.0–11.0%，換算成等效功耗多出 39–127 W（800 W 的 127 W 是在 800 與 1000 W 之間直線內插，偏高估），總耗電量少 4.1–9.3%。在預設的 1000 W 上限，調參後的原生仍有 75% 的取樣撞上限（SM 平均 1936 MHz），截斷後只剩 7%（SM 1977 MHz），所以也快 3.9%，模組功耗從 930 W 降到 886 W，每次求解少耗電 7.5%。未調參的設定（N = 262144，Slurm 工作 1670）在 400–800 W 快 3.4–8.3%、1000 W 快 0.7%、省電 7.7%，結論相同。

實際效能（8 節點、N = 286720、調參後，單位 TFLOP/s，兩次的平均；Slurm 工作 3315；gh200/results/e3_tuned_summary.csv）：

每節點模組上限	原生	保留 36 位	加速	等效多出的上限	每次求解的耗電
400 W	225.0	249.9	+11.0%	39 W	−9.3%
500 W	288.0	300.7	+4.4%	42 W	−4.1%
600 W	318.0	334.0	+5.0%	57 W	−5.1%
700 W	346.2	362.9	+4.8%	68 W	−4.8%
800 W	370.8	385.6	+4.0%	127 W	−4.2%

每節點模組上限	原生	保留 36 位	加速	等效多出的上限	每次求解的耗電
1000 W	393.9	409.2	+3.9%	—	−7.5%

8 顆 GH200 的 FP64 tensor core 規格峰值合計約 536 TFLOP/s（每顆 67 TFLOP/s [NVIDIAGH200]），調參後的原生 HPL 在 1000 W 達到約 73%（重測兩次平均 393.9，調參時單次最高 398.8 TFLOP/s），保留 36 位達到 409.2 TFLOP/s（研究用途，不是正式成績）。功耗上限壓得越低，效能掉得越多，截斷幫忙把掉掉的部分搶回來。保留 28 位可以再多 0–5 個百分點（例如 700 W 時 372.3 TFLOP/s），但它在這個 N 的殘差已到 15.4（門檻 16）、在 N = 196608 更是失敗，所以不當作操作點。

殘差與失效邊界。 HPL 最後會檢查縮放殘差是否小於 16。殘差會隨矩陣大小 N 變化，所以我們在 1000 W 用三組 N 檢查（每組 2 次，殘差完全重現），再加上調參後的 N = 286720（上限掃描的 12 次，殘差也完全重現）：

保留位數	N = 131072	N = 196608	N = 262144	N = 286720	判定
原生（53 位）	1.5×10^{-4}	1.3×10^{-4}	2.6×10^{-5}	7.6×10^{-5}	通過
36	0.16	0.072	0.042	0.038	通過，離門檻約百倍（最小 98 倍）
28	13.97	**18.09**	14.70	15.40	N = 196608 失敗，其餘都接近門檻
24	456	462	91	—	全部失敗

從 36 位到 28 位少了 8 位，殘差放大 86–410 倍，量級與 $2^8 = 256$ 相符。這代表 28 位離門檻不到 1 位，換一個 N 就可能失敗；36 位則有約 7 位的餘裕，所以我們以 36 位作為 HPL 的操作點。

HPL-MxP。 HPL-MxP 用 BF16 做 LU 分解、FP64 迭代修正。我們在每次寫回 BF16 時歸零，8 節點、N = 491520。這裡用的測試矩陣是對角優勢矩陣（對角元等於該列非對角元絕對值的和），對角 tile 用近似分解（--approx-diag），迭代修正用 Richardson 法加 FP32 預條件子。依 HPL-MxP 的規則，嚴格對角優勢的矩陣只能用於測試 [HPLMxPRules]；在強對角優勢的矩陣上，很粗的 L、U 仍是好的預條件子，所以下面「保留 3 位仍穩定」的結論不能直接外推到正式允許的矩陣。

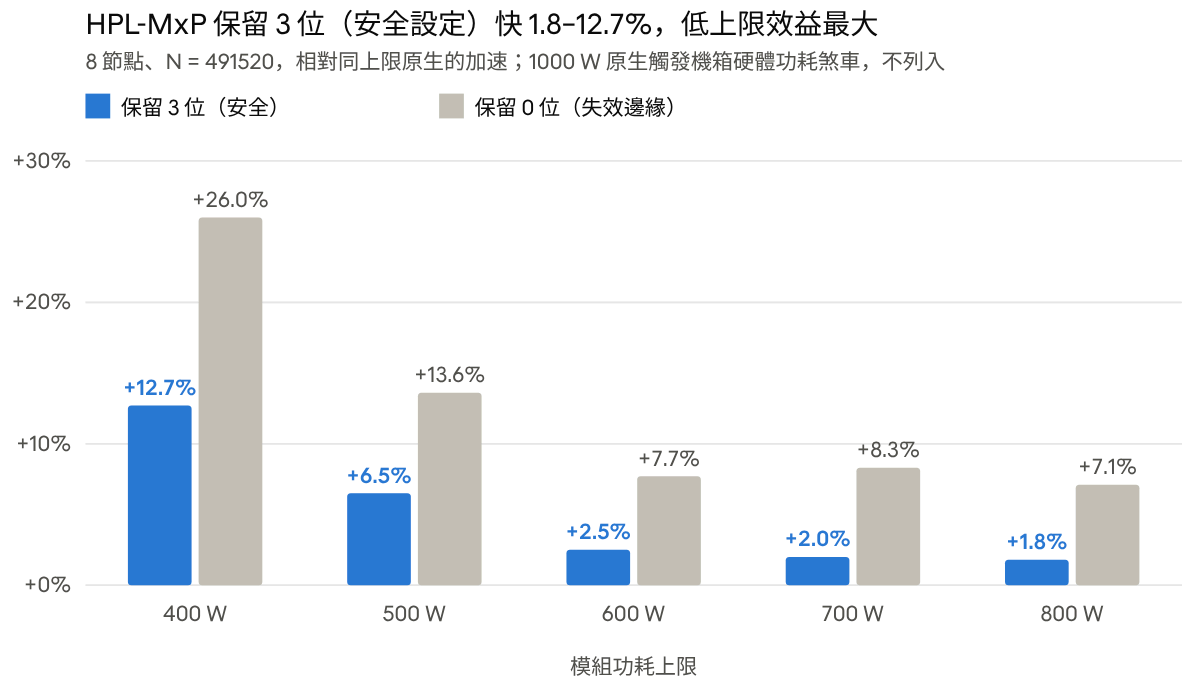


圖 5.4　HPL-MxP 用對角優勢的測試矩陣（規則只允許用於測試）；Slurm 工作 1671；gh200/results/e2_summary.csv

保留 3 位在 400–800 W 快 1.8–12.7%（等效多出 34–45 W、耗電量少 3.4–10.7%），在 N = 245760–491520 的四組大小都穩定：殘差 1.4–3.8、固定 3 次修正。每個上限的修正次數都從原生的 1 次變成 3 次；LU 本身的加速從 400 W 的 16.0% 降到 800 W 的 6.0%，扣掉多出的修正時間後，600 W 以上只剩約 2%。保留 0 位雖然快 7.1–26.0%，但 N = 491520 的殘差已到 10.6–11.1（門檻 16），在 N = 245760 時修正次數暴增到 52 次、效能掉 70%，所以只是失效邊界。

實際效能（8 節點、N = 491520，單位 TFLOP/s，報告值含迭代修正，兩次的平均；gh200/results/e2_summary.csv）：

每節點模組上限	原生 BF16	保留 3 位	加速	等效多出的上限	每次求解的耗電
400 W	2076	2340	+12.7%	35 W	−10.7%
500 W	2823	3007	+6.5%	45 W	−6.6%
600 W	3229	3310	+2.5%	37 W	−3.7%
700 W	3449	3519	+2.0%	34 W	−4.7%
800 W	3655	3720	+1.8%	38 W	−3.4%
1000 W	3015＊	3976	—	—	—

＊原生 HPL-MxP 在 1000 W 會觸發機箱的硬體 power brake，效能被拉低，不計入比較；等效上限改用同一組節點上另一個工作（1675）量的 900 W 原生點（3830 TFLOP/s，兩次平均）換算。保留 0 位在這個 N 上更快（例如 400 W 時 2616 TFLOP/s），但在較小的 N 上修正次數暴

增，所以不當作操作點。

一個副產品發現：風扇全開時，原生 HPL-MxP 在 1000 W 會觸發機箱電源的硬體功耗煞車（每次 4–10 秒），反而比 800 W 慢；歸零版本的尖峰功耗較低：有開計數器的診斷工作中，保留 0 位的版本沒有觸發。

小結： 在有殘差檢查把關的完整基準上，功耗受限時尾數歸零都帶來正面收益；但保留位數必須對多組問題大小驗證，選擇有餘裕的操作點；HPL-MxP 的結果還限於對角優勢的測試矩陣。

5.4 科學軟體：Quantum ESPRESSO 與 PySCF

需要幾位？ 我們用攔截函式庫截斷所有 FP64 矩陣乘法，看物理結果（總能量）在多少位時開始偏離：

系統	結果
Quantum ESPRESSO 金表面（112 原子）	最近捨入 24 位誤差 8.1×10^{-6} Ry、20 位 5.5×10^{-5} Ry、16 位發散
Quantum ESPRESSO 矽（Si 1000）	28 位誤差 3.5×10^{-7} Ry；20 位是失效邊緣（一次收斂、一次中止）
PySCF 纈胺黴素 DFT（門檻 10^{-7}）	32～40 位都與原生同樣 12 圈收斂；32 位誤差 1.2×10^{-5} Eh（約 0.007 kcal/mol）
PySCF 紫杉醇 HF+MP2	往零截斷 44 位誤差 5.6×10^{-8} Eh；36 位 2.7×10^{-5} Eh

平面波密度泛函只需要 20–28 位，高斯基底的量子化學需要 32–36 位。後者與 Dawson 等人以模擬得到的「約 37 位有效尾數即足夠」一致 [Dawson2024]。

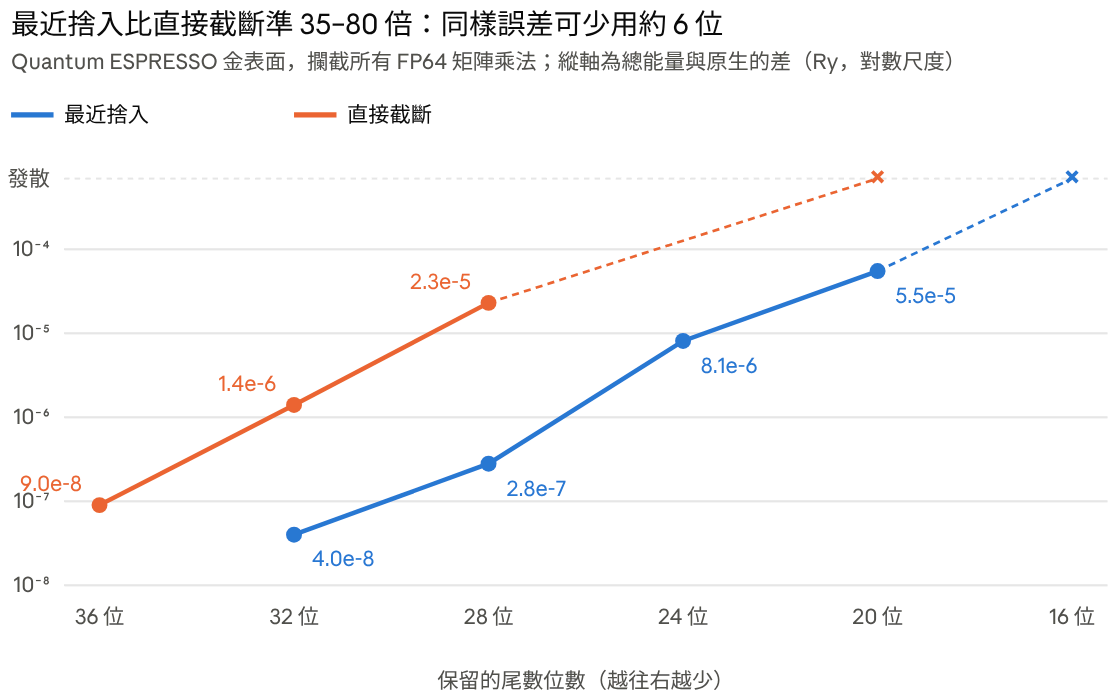


圖 5.5　Slurm 工作 1729、1738（金表面攔截掃描）

捨入方式很關鍵。 最近捨入的優勢遠大於單次矩陣乘法的差距（誤差係數 α 只差約 2.6 倍）：截斷永遠把數值往 0 拉，誤差有固定方向，在自洽迭代裡一路累積；最近捨入沒有偏差，誤差會互相抵銷。

效能與耗電。 這些程式平常用不到 1000 W，所以在會碰到上限的 450–550 W 量：

做法	系統與上限	時間	耗電量	總能量誤差
依收斂門檻自動調整位元	金表面，450 W	**−4.2 ± 0.7%**	**−3.9 ± 1.1%**	**0（印出的每一位都相同）**
依收斂門檻自動調整位元	矽，550 W	**−2.4 ± 0.5%**	**−2.5 ± 0.6%**	**0（印出的每一位都相同）**
程式內固定 24 位	金表面，450 W	−2.7 ± 1.1%	−2.7 ± 1.6%	3×10^{-8} Ry
程式內固定 28 位	矽，550 W	−2.3 ± 0.6%	−2.3 ± 0.6%	$< 10^{-8}$ Ry
攔截所有 GEMM，最近捨入 28 位（上限）	矽，550 W	−6.2%	−6.4%	3.5×10^{-7} Ry

前四列是程式內的做法，與未修改的原生程式交錯跑 6 次（成對），數字是 6 對的平均 ± 95% t 區間。每個工作的第一次原生執行偏慢（暖機，例如金表面 96.9 秒、其餘 94.2–94.7 秒），含它的那一對收益偏大；排除這一對，自動調整在金表面是 −3.9%、矽是 −2.3%。金表面上，自動調整在 6 對中每一對都比固定 24 位快 0.6–1.8 個百分點；矽上也每對都略快於固定 28 位，但只差 0.18 ± 0.14 個百分點。所以自動調整至少追平事後才找得到的最佳固定位數，而且不用事先試。最後一列攔截所有矩陣乘法，是「如果函式庫內建截斷」的上限（與同樣付出複製成本的基準比，各跑一次）。

自動調整的規則讓位元數隨收斂從 16 升到 39 位，讓截斷誤差大致不超過 Davidson 當圈本來就容許的誤差、由後續迭代修正，所以總能量在印出的所有位數（10^{-8} Ry）都與原生相同，SCF 圈數也相同。不過金表面的收益有一部分不是功耗效應：自動調整讓 Davidson 少做了約 7–9%（h_psi 呼叫 311 對 336 次）；在一次上限被外部重設為 1000 W、不受功耗限制的執行中（工作 2003，時脈相同），它仍比原生與固定 24 位快約 2%（73.5–73.7 對 74.8–75.1 秒），和 450 W 時領先固定 24 位的幅度相同。所以金表面 −4.2% 中約一半來自迭代變少，原因尚未查明；矽上沒有這個現象（206 對 205 次），−2.4% 可視為功耗效應。攔截版的數字是對「複製但不截斷」的基準比，代表「如果函式庫內建截斷」的上限；它本身約有 5% 的複製成本。

單精度行不行？ Quantum ESPRESSO 的運算元值域放得進 FP32，所以我們也試了把所有 GEMM 改成 FP32、TF32 或 3xTF32（3xTF32 是攔截工具內自寫的實作：拆成兩個 TF32、做三次 FP32 累加的 tensor core 乘法）：金表面的 FP32 在第 26 圈、TF32 與 3xTF32 在第 2 圈中止；矽的 FP32 在第 14 圈、3xTF32 在第 9 圈中止，積分電荷也明顯偏離。原因在第 5.5 節說明。這是把所有 GEMM 都換掉的極端做法；只在部分步驟用 FP32 的混合精度（例如 Woo 等人 [Woo2023]）不在此限。

小結： 科學軟體需要 20–36 位，在功耗受限時能拿到 2–6% 的時間與耗電。最有價值的是「跟著收斂門檻調整位元數」：總能量印出的每一位都不變，而且不需要事先知道安全的位數。

5.5 與 FP64 模擬及單精度的分界

「用較少位元算 FP64 矩陣乘法」還有另一條路：用 INT8 tensor core 模擬 FP64。我們在同一節點、同一功耗上限下，把誤差對齊到同一級後比速度。小矩陣與 CPU 精確值對比確認：同樣的位元數下，截斷與模擬的誤差在同一級。

功耗上限會改變勝負。 在方陣上：

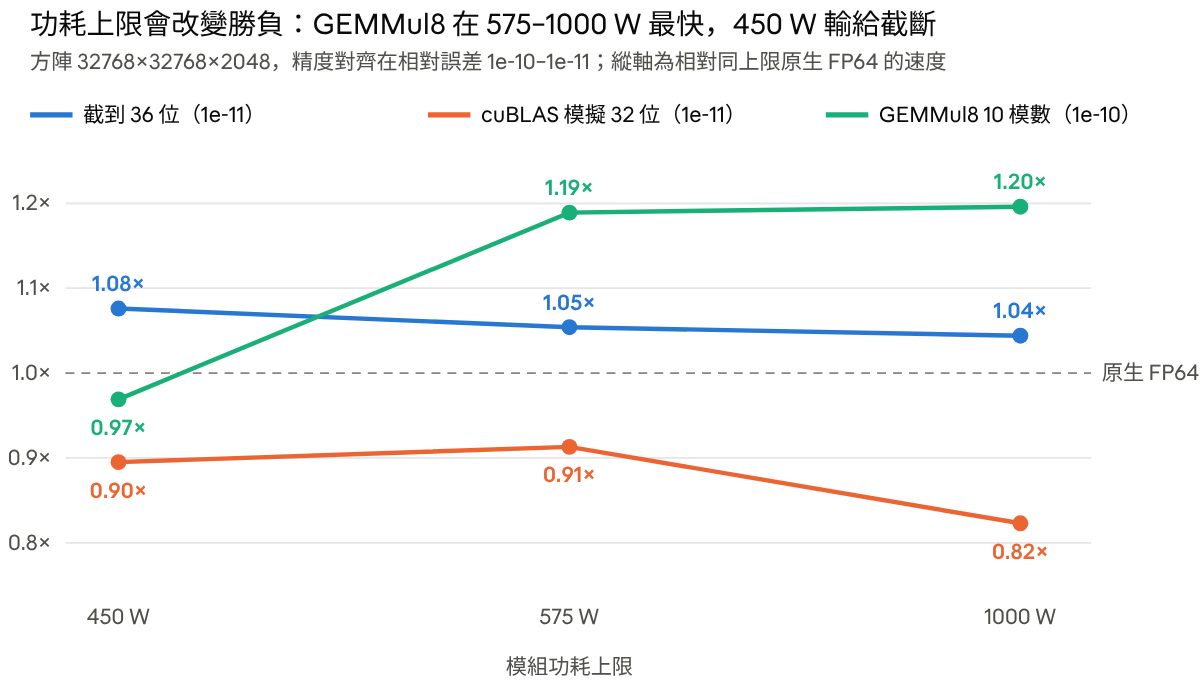


圖 5.6　Slurm 工作 1894；gh200/results/m3_ozaki2_tf32x3_summary.txt

GEMMul8 很耗電：1000 W 時就用到約 970 W，450 W 時時脈掉到 650–720 MHz，反而輸給截斷。cuBLAS 模擬在 1000 W 只用約 706 W。反過來，cuBLAS 模擬在方陣上隨上限降低而更有優勢（28 位時從 1000 W 的 1.07 倍升到 450 W 的 1.20 倍）。所以上限收緊對誰有利，要看模擬本身耗多少電；截斷在每個上限都比原生快，但幅度不隨上限單調變化（第 5.2 節）。在較粗的精度等級（10^{-9}）則是模擬較快；cuBLAS 模擬以約 8 位為一片，精度是階梯式的，截斷則是 1 位一格。

矩陣形狀決定勝負。 我們攔截 PySCF 纈胺黴素計算的全部 45,574 次 GEMM，發現兩種形狀各佔 35% 運算量：

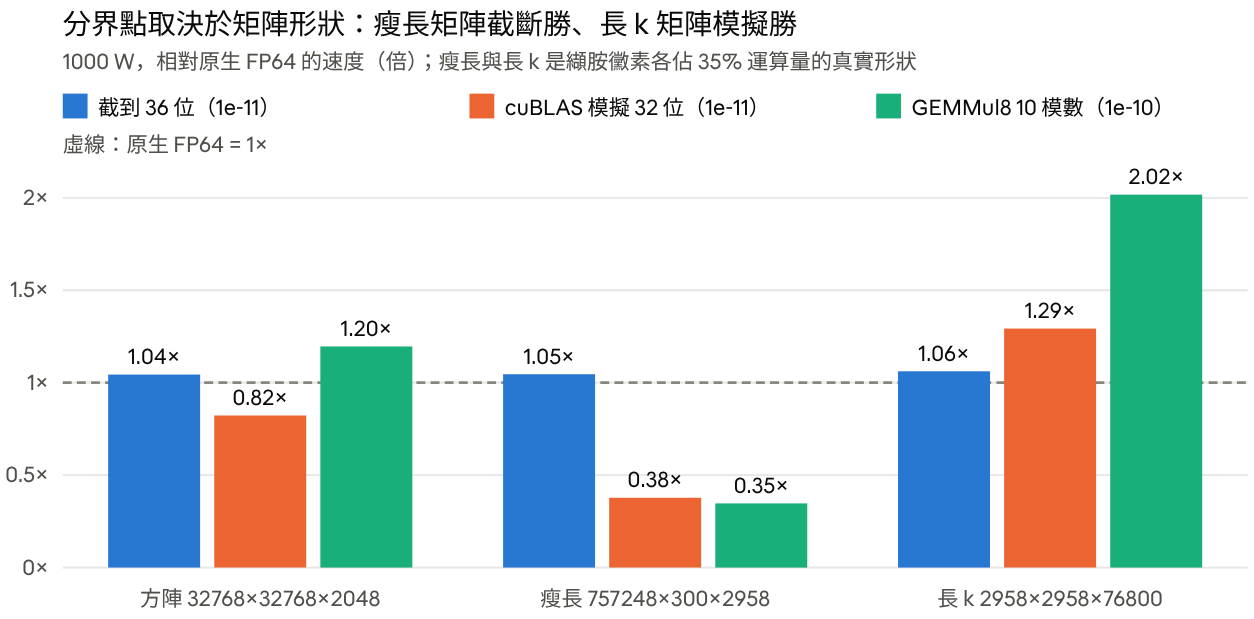


圖 5.7　Slurm 工作 1894、1896、1897；gh200/results/m3_realshape_summary.txt

模擬需要先把輸入拆片、最後再合併，這部分的成本與矩陣元素數（約 mk + kn + mn）成正比，而乘法本身是 mnk。n = 300 的瘦長矩陣上，拆片成本和乘法一樣大，模擬只有原生的 0.47–0.57 倍速度；k = 76800 的長 k 矩陣上，拆片成本被均攤，INT8 的高運算量就顯現出來。截斷沒有拆片成本，在一個上限下對任何形狀的加速都差不多（例如 1000 W 兩種形狀都快約 5–6%）。最佳做法是依形狀選擇：瘦長的截斷、長 k 的模擬。

分界點掃描。 為了把「依形狀選擇」變成可以用的規則，我們固定 m = 8192，n 取 256／1024／4096／8192、k 取 256 到 81920，在 1000／575／450 W 各量兩輪，共 480 筆。

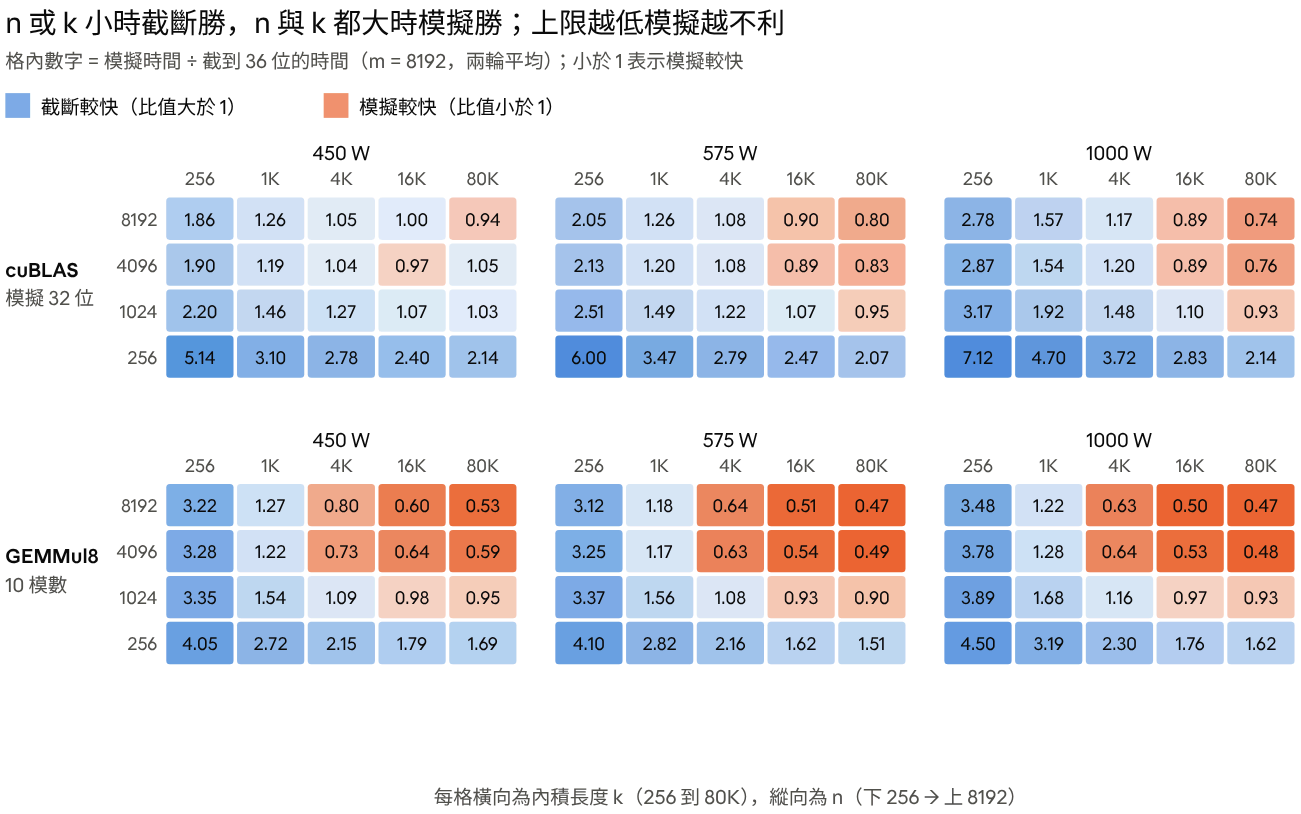


圖 5.8　Slurm 工作 2005–2008（480 筆）；gh200/results/crossover_*.csv

截斷保留 36 位在所有 60 組（形狀 × 上限）都比原生快（450 W 3.8–11.9%、575 W 3.6–5.9%、1000 W 0.5–4.8%）。模擬只在 n 與 k 都大時較快：cuBLAS 模擬需要 n ≥ 4096 且 k ≥ 16384，勝出的形狀在 1000／575／450 W 分別有 5／5／2 種；但在模擬本身吃不滿電的形狀上（例如 n = 8192、k = 4096，模擬對截斷的時間比從 1.17 降到 1.05），上限越低反而對模擬越有利，方向取決於模擬本身是否受功耗限制；GEMMul8 需要 n ≥ 4096 且 k ≥ 4096，在那裡時間只要截斷的 0.47–0.80 倍，但精度低一個量級（10^{-10} 對 10^{-11}）。以「模擬時間 ÷ 截斷時間 = A + B·(1/m + 1/n) + C·(1/k)」擬合，GEMMul8 的決定係數 R^2 達 0.98–0.99；只用單一的有效大小 1/(1/m + 1/n + 1/k) 則只有 0.86–0.89。這代表 GEMMul8 的輸入拆片與輸出合併是兩種不同的成本。cuBLAS 模擬的 R^2 只有 0.88–0.89，與單一有效大小的模型差不多，還有模型沒捕捉到的因素（例如不同形狀選到不同核心）。四個 n 值分別在不同節點上量；每格的比值是同節點配對，但跨形狀、跨上限的趨勢可能混入節點差異。

這條公式只用 m = 8192 的資料擬合。我們拿先前量過、沒有參與擬合的兩種纈胺黴素真實形狀來檢驗（1000 W 與 575 W、兩種模擬方法，共 8 個案例）：瘦長 757248×300×2958 預測截斷勝、長 k 2958×2958×76800 預測模擬勝，8 個案例的勝負全部判對。倍數本身的誤差 cuBLAS 模擬為 6–15%，GEMMul8 為 24–37%，後者較大是因為瘦長矩陣的 m = 757248 遠超出擬合範圍。對使用者來說，判斷「該用哪一種」只需要勝負正確，所以這個精度已經夠用。不過這 8 個案例都離分界很遠（模擬的時間不是截斷的一半到八成，就是兩倍以上），只是寬鬆的檢驗；公式只用 m = 8192 擬合，對 m 的依賴也沒有驗證。

FP32 與 3xTF32 為什麼不行。 3xTF32（自寫的實作，不是 CUTLASS 的版本）在方陣上最快，但它的誤差隨內積長度 k 成長；截斷後的 FP64 仍以 FP64 累加，誤差與 k 無關。

內積長度 k	3xTF32	TF32	FP32
2048	4.8×10^{-6}	2.6×10^{-4}	8.1×10^{-7}
16384	3.8×10^{-5}	—	2.3×10^{-6}
95386（金表面的平面波數）	5.6×10^{-5}	2.7×10^{-4}	2.8×10^{-6}
375286（矽的平面波數）	2.2×10^{-4}	3.4×10^{-4}	1.9×10^{-6}

在 4096×4096 上，k 每增 8 倍，3xTF32 的誤差就增 7.5–7.9 倍，這與 tensor core 以 FP32 累加時朝零捨入的既有觀察一致 [Fasi2021]。Quantum ESPRESSO 的 GEMM 內積長度就是平面波數（數十萬），所以 3xTF32 退化成與 TF32 同級，實測無法收斂（第 5.4 節）。

小結： 截斷與模擬沒有誰全勝，分界點由需要的精度、矩陣形狀與功耗上限決定；兩者應依形狀互補使用。把所有 GEMM 換成單精度（FP32、3xTF32），在長內積上不是 FP64 截斷的替代品。

5.6 非矩陣乘法的 FP64 程式

Ozaki 類模擬只適用於稠密矩陣乘法，所以如果尾數歸零在其他 FP64 運算上也有效，它就有模擬取代不了的空間。我們先在核心層級測試：只用一般 FP64 運算單元的乘加迴圈、記憶體複製與 triad、N 體重力計算，以及一個 CG 稀疏求解器。

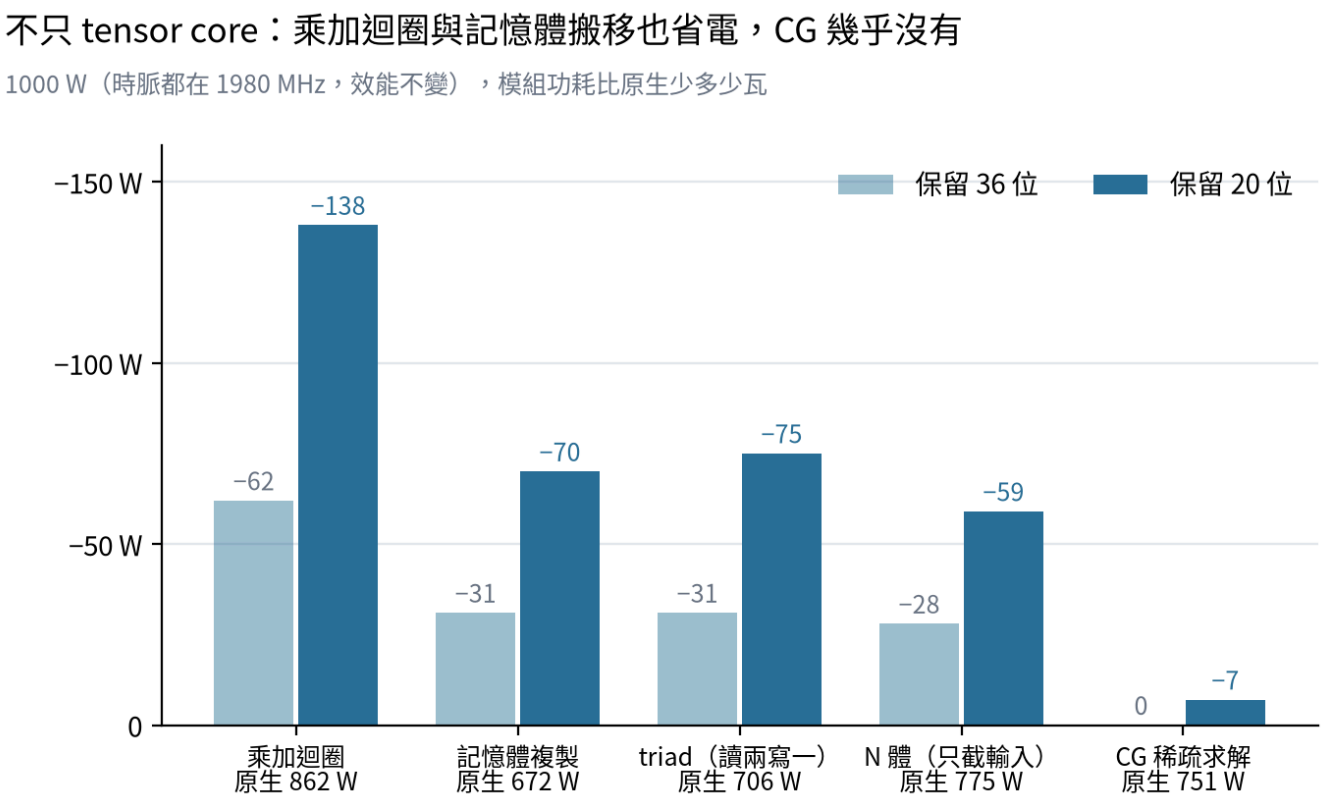


圖 5.9　Slurm 工作 1815、1858、1860；gh200/results/m4_*_summary.txt

在低功耗上限下，這些省電轉成加速：乘加迴圈在 500 W 快 4.8–9.0%（36–20 位），記憶體複製在 450 W 快 7.7–14.0%。但 N 體只快 1–4%：它第一步就是座標相減 $dx = x_j - x_i$，差值的尾數又是滿的，後面約 20 個運算照常翻轉。這說明效益隨「計算深度」遞減：只有直接讀取截斷資料的運算受惠。CG 幾乎沒有效益，因為被截的矩陣值只佔每圈記憶體流量的一部分，索引與向量不能截。

第三方真實軟體。 為了確認核心層級的結果能否延伸到真實程式，我們在三個不以矩陣乘法為主的 FP64 GPU 程式中，於設定階段對「儲存後每步被乘進去的係數陣列」歸零一次（不設定時與原版逐位相同）：

軟體	被截的係數	係數佔記憶體流量	結果
MFEM（免組裝有限元素，3827 萬未知數）	每個求積點的幾何與係數資料	約 43%	450 W 快 0.9%（36 位）／2.0%（20 位），550 W 快 0.7／1.4%；解的相對變化 ⩽ 2.3×10^{-11}

軟體	被截的係數	係數佔記憶體流量	結果
SW4（地震波，1.26 億格點）	材料參數 ρ、μ、λ	約 9%	350 W 快 0.6%（36 位）／1.3%（20 位），450 W 快 0.3／0.5%；波形最大相對差 ≤ 3.4×10^{-6}
LAMMPS（EAM 分子動力學）	位能表	約 0%（常駐快取）	各上限 0.0–0.4%，落在雜訊內（負面對照成立）

MFEM 的係數佔了近一半的記憶體流量、直接進乘法器，收益仍只有 1–2%。三者的收益順序（MFEM ≥ SW4 > LAMMPS）與係數佔記憶體流量的比例一致。

小結： 尾數歸零的省電不限於 tensor core，一般 FP64 運算單元與記憶體搬移也省；但在真實程式中，被截斷的資料只是耗電的一部分，核心層級的 7–18% 會被其餘滿尾數的運算稀釋到 1–2%。

5.7 AI 工作負載

訓練：Llama 2 70B LoRA 微調。 MLPerf Training 比的是「達到目標品質要花多少時間」（驗證損失 ≤ 0.925）。基底權重已經是 FP8（只有 3 個尾數位元），我們在載入時再截一次，8 節點。

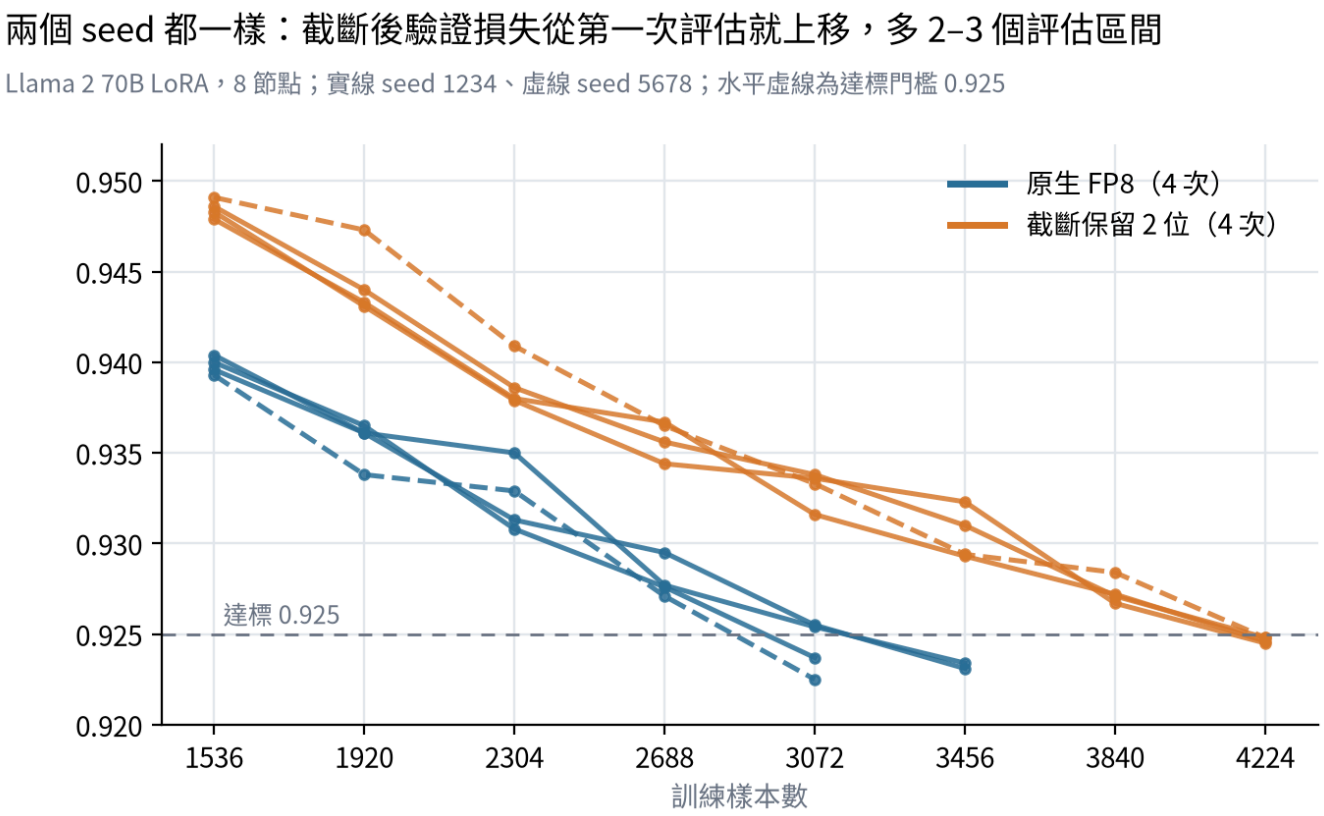


圖 5.10　Slurm 工作 1732、1736、1760 起、1901、1904、1990、1993；分層截斷 2011、2020、2027 只列在內文；gh200/logs/lora

兩個亂數種子、三組配對的結果一致：截斷 2 位的驗證損失從第一次評估就高 0.008–0.010，要多花 2–3 個評估區間才達標（例如 seed 5678：原生 3072 個樣本、22.74 分，截斷 4224 個樣本、31.56 分）；而每步只快約 1%。截斷在載入時就削弱了基底模型，LoRA 補不完。最近捨入與隨機捨入也沒有讓情況變好（分別要 5376 與 4224 個樣本）。只截部分層也一樣：只截 MLP 要 4224 個樣本，與全截相同；只截注意力要 3840 個，退化在雜訊內，但注意力只佔線性層權重約 18%，可以省的本來就很少。只截注意力以完全相同的設定再跑一次，第一次評估幾乎一樣（0.9428 vs 0.9425），達標點卻差一個區間（3840 vs 4224），所以我們以第一次評估的差距作為主要證據。結論是：對已經是 FP8 的訓練，尾數已經沒有可以再省的空間。

推論：BF16 截斷 vs FP8。 我們找兩種可能的機會：比 FP8 還快、精度又不會差太多的做法；或 BF16 加尾數截斷比 FP8 好的場景。

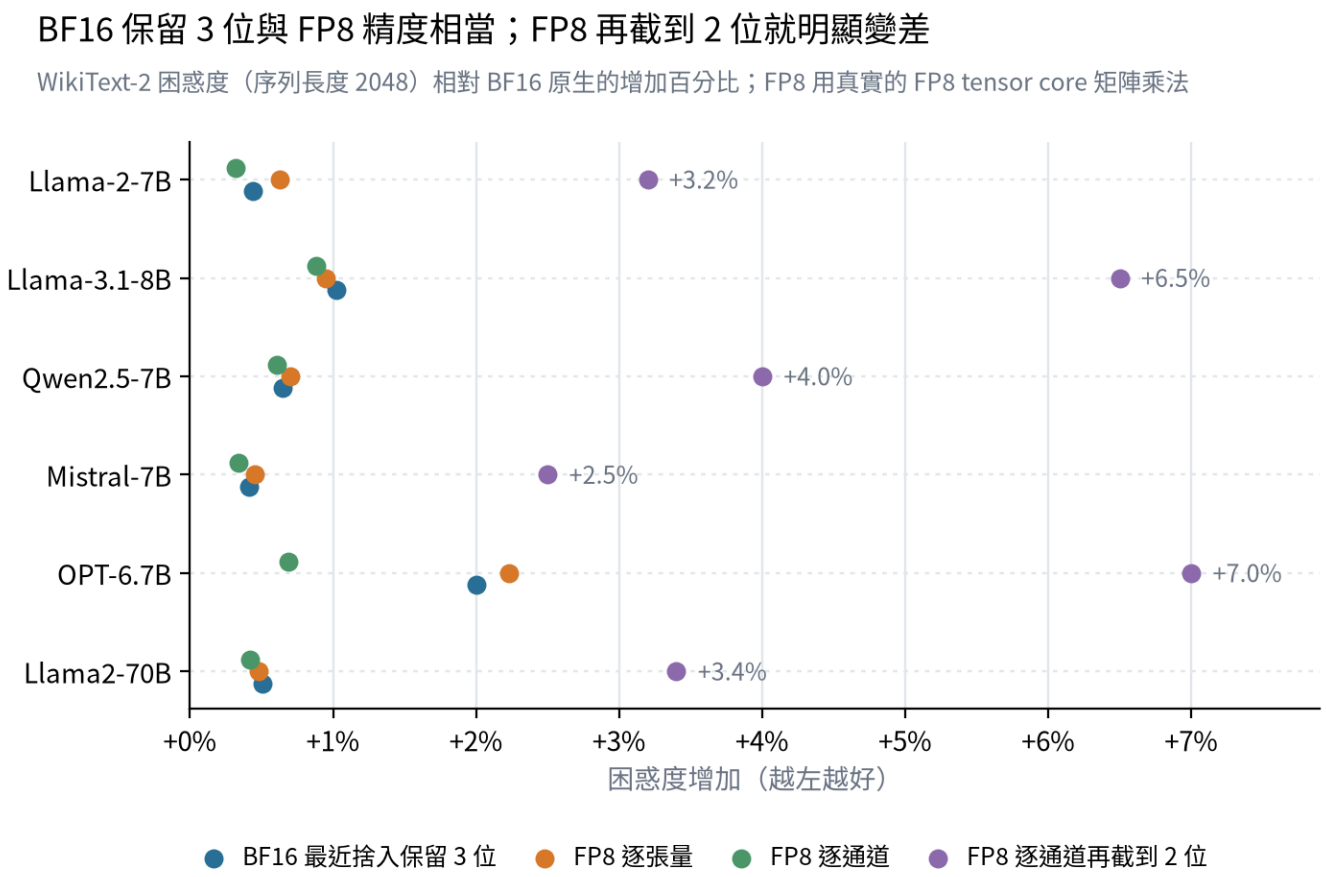


圖 5.11　gh200/results/infer_ppl.csv（70B 為逐層評估）

以最近捨入保留 3 位的 BF16 與 FP8 尾數位數相同，矩陣乘法誤差也相當（均勻資料上 3.3% 對 3.6–3.9%；截斷要保留 4 位才接近，約 4.4%）。我們接著在文獻中 FP8 最容易出問題的地方比較兩者，結果都打平：

場景	BF16 最近捨入保留 3 位	FP8
注意力（2k／8k 上下文困惑度）	+0.49%／+1.06%	+0.52%／+1.08%
長上下文鑰匙檢索（8k–32k）	100%	100%
BERT-large SQuAD（MLPerf 99.9% 門檻）	通過	通過
CogVideoX-2B 影片（對原生的 PSNR）	20–29 dB	18–38 dB

而 FP8 的矩陣乘法快 1.7–1.9 倍（575 W 實測）。在不改用結構化稀疏或 INT4 權重的前提下，更快的路只剩在 FP8 之上再截：以 Llama2-70B 真實權重，兩邊都截到 2 位快 3.2–8.2%，但困惑度增加 3.4%，速度與精度大致 1 比 1 交換；decode 階段的省電也只有 1.5–5.5%。

小結： 在 Hopper 上，FP8 已經拿走了「減少尾數位元」幾乎全部的好處。尾數歸零的價值在 FP64 科學計算，而不在已經用 FP8 的 AI。

5.8 高耗電的真實應用：DF-MP2

第 5.4 節的科學軟體在 1000 W 下平均用不到上限（金表面約 535–600 W、矽 1000 約 800 W），要壓低功耗上限才看得到明顯的加速。我們因此找了一個高耗電的應用：GPU4PySCF 的密度擬合二階微擾理論（DF-MP2）。它的 MP2 階段 GPU 功耗約 800 W（1000 W 時模組約 870 W，只有部分時間撞上限）、矩陣乘法佔 GPU 時間 78%，而且幾乎都在同一種收縮：把密度擬合張量的兩個切片相乘（FP64 DGEMM）。截斷做在程式內：張量建好後原地截一次尾數，之後所有矩陣乘法都受惠，不需要複製。同一個程序先做一次 HF，再依序跑各種 MP2 版本（紫杉醇 def2-TZVP，每個上限交錯跑兩輪，截斷變體的兩輪差距都小於 0.6 個百分點）：

上限	原生	截斷 32 位（誤差 1e-11 Eh）	截斷 20 位（1e-8 Eh）	內建 FP32（3–4e-8 Eh）	cuBLAS 模擬 32 位（2e-8 Eh）
1000 W	39.5 s	+0.3%，少 61–70 W、省電 8%	+0.8%，少 106–115 W、省電 14%	+2.6／+11.5%（兩輪差距大）	慢 36%
700 W	44.1 s	+5.3%	+7.6%	+18%	慢 28%
500 W	52.0 s	+4.9%	+8.3%	+15%	慢 18%

截斷在功耗受限時快 4.9–8.3%，1000 W 時速度幾乎不變（+0.3–0.8%）但少用 61–115 W，和 HPL 同級。GPU4PySCF 內建的 FP32 更快，誤差也在化學精度內，因為這個張量的指數跨度很窄（整體 27、列內中位數 18 個 2 的次方），FP32 放得下。cuBLAS 模擬在這個形狀一律較慢。

小結： 在高耗電的真實應用上，截斷拿到與 HPL 同級的收益；但資料跨度窄時，換格式（FP32）可能更快。下一節說明什麼時候換格式會出問題。

5.9 指數跨度：哪些資料只能用截斷

截斷保留每個元素自己的 11 位指數，只改尾數；換格式的做法都要先處理指數：FP32、FP8 以整個張量的範圍為準，INT8 模擬（Ozaki 類）以每一列的最大值為基準切片，比最大值小 2^{-k} 以上的元素在保留 k 位時會被丟掉。所以「數值大小差多少」會限制整個矩陣共用縮放的換格式做法，而截斷不受影響。

合成實驗。 讓矩陣乘法（8192×8192×16384，這個形狀在跨度小時是模擬較快）的每個元素，指數在 S 個 2 的次方內均勻分佈：

跨度變大時，截斷的誤差不變，模擬變差、FP32 溢位

8192×8192×16384，每個元素的指數在 S 個 2 的次方內均勻分佈；縱軸為逐元素相對誤差的中位數

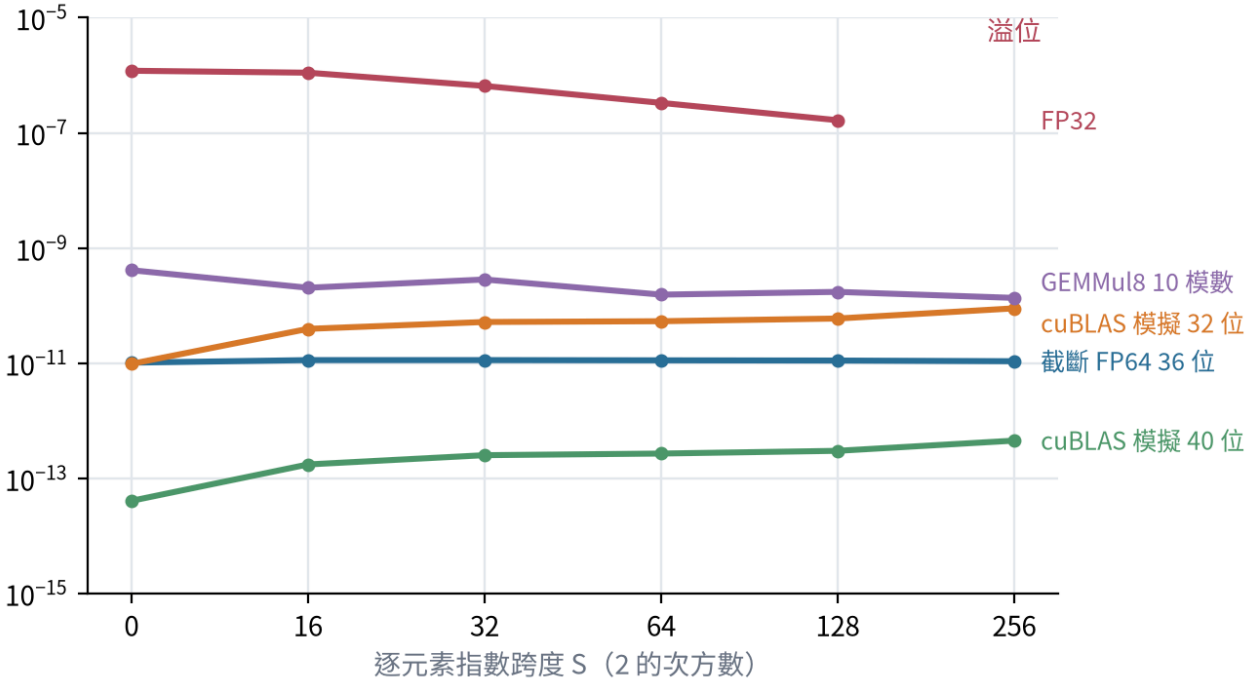


圖 5.12　Slurm 工作 2034–2036（1000／575 W，兩輪）；gh200/results/range_sweep.csv

截斷 36 位的誤差與加速（575 W 快 5.7–6.3%、1000 W 快 4.4–4.5%）完全不受跨度影響。固定位數的 cuBLAS 模擬精度變差 4–9 倍（相當於少了 2–3 位），而模擬以約 8 位為一片，要多付一整片才追得上；保證 FP64 精度的動態模式所需位數從 58 升到 301 位，超過上限就改用原生 FP64，但連同判斷跨度的額外工作，時間是截斷的 1.6–2.6 倍；FP32 在跨度 256 時溢位。GEMMul8 在這種均勻分佈的資料上幾乎不受影響。另外，若只是每列各自一個比例（整體寬、列內窄），模擬完全不受影響，但 FP32 仍然溢位。

真實應用。 我們在攔截工具加上逐列跨度統計（每個矩陣乘法抽樣 256 列／行，以合成資料驗證與理論值相符），量出各應用「比所在列最大值小 2^{-32} 以上」的元素比例，並在各應用上直接比較截斷與模擬：

應用	列內小 2^{-32} 以上的元素	截斷 36 位	cuBLAS 模擬固定 36 位（5 片）	GEMMul8 10 模數
PySCF 纈胺黴素 DFT	24–44%（格點上的原子軌域矩陣達 87%）	9.6e-7 Eh	**0.0525 Eh（約 33 kcal/mol）**	發散
QE 矽	0.24–0.54%	0 Ry	6e-8 Ry	GPU 記憶體不足
QE 金表面	0.008–0.02%	0 Ry	5e-8 Ry	3.7e-7 Ry

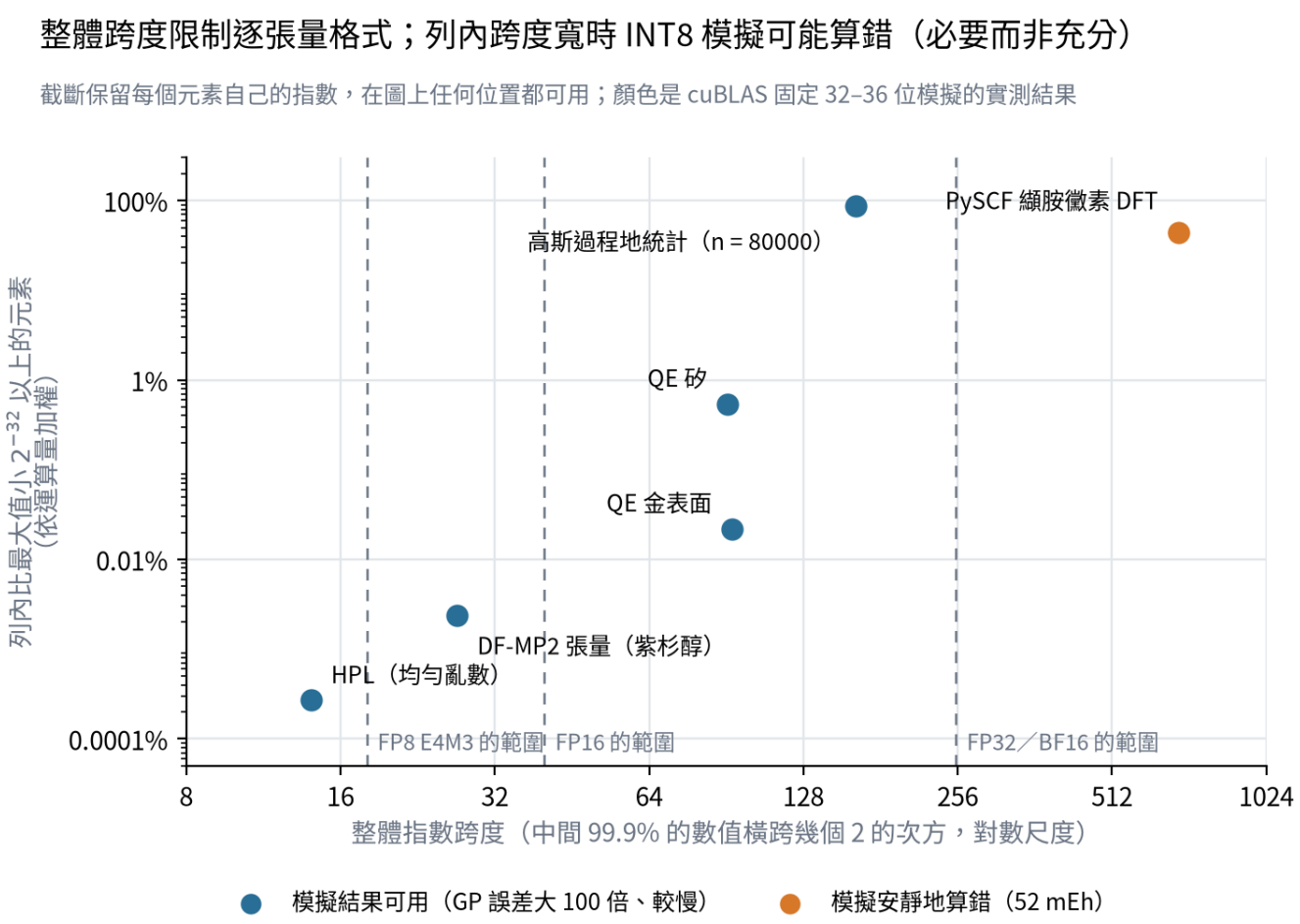


圖 5.13　gh200/results/range_map_points.csv（每個點的來源列在檔內；HPL 與高斯過程的點來自工作 3291、3304、3305）

同樣是模擬 36 位：在列內跨度窄的 QE 上完全正常，在列內跨度寬的 DFT 上，SCF 照常收斂、沒有任何警告，能量卻錯了 52 mEh；截斷在兩邊都正常。模擬加到 40 位（6 片）就算對了（誤差 8×10^{-8} Eh），但時間是截斷 36 位的約 1.5 倍（第 5.10 節）；出錯的是使用者指定的固定位數模式。cuBLAS 的自動模式（依指數跨度決定位數，上限設 79 位）則沒有算錯（誤差 4×10^{-9} Eh）：89% 的運算量需要 80 位以上而改用原生 FP64，其餘用 72–79 位模擬，加上判斷跨度的

額外工作，時間是原生的 2.1 倍（1000 W；Slurm 工作 3292）。所以寬跨度資料上的模擬不是不能用，而是不划算。反過來，跨度很窄的 HPL（依運算量加權只有 2.7×10^{-6} 的元素比列最大值小 2^{-32} 以上）用固定 36 位模擬也通過殘差檢查（0.197；截斷 36 位為 0.057），但單 GPU、N = 81920 時只有原生 65% 的速度（工作 3304、3305）。QE 金表面的跨度統計取自只跑 2 圈的基準輸入，能量誤差取自完整收斂的輸入。合成的均勻分佈低估了這個問題，因為真實的原子軌域值隨距離指數衰減，同一列裡很多元素比最大值小很多。

小結： 指數跨度會限制換格式的做法：逐張量縮放的格式看整體跨度，逐列切片的 INT8 模擬看列內跨度。跨度寬時，整個矩陣共用縮放的格式或固定位數的模擬可能出錯，截斷則不受影響；但列內跨度寬只是模擬出錯的必要條件（第 5.10 節的高斯過程就沒有錯），以 tile 為單位換格式也能處理這類資料。列內跨度可以用攔截工具事先量出來，作為選擇方法時的風險指標。

5.10 跨度寬又吃滿電：高斯過程地統計

前兩節的應用不是跨度寬、就是耗電高，沒有兩者兼具：幾種 DFT 變體跨度都寬，但在 1000 W 只用到 650–690 W。不過只要功耗上限低於它們的用電，寬跨度的 DFT 也會受限：纈胺黴素（格點等級 6）在 550／450 W 時，截斷 36 位快 3.3–3.5%、省電 2–3%，誤差 10^{-6} Eh；模擬要用到 40 位才不會算錯，時間是截斷的 1.5 倍。

為了找到兩者兼具的例子，我們轉向地統計的高斯過程最大概似估計（ExaGeoStat 這類應用）。每次評估對數概似都要對 n×n 的 Matérn 共變異矩陣做 Cholesky 分解，計算量和 HPL 一樣幾乎全是矩陣乘法；共變異隨距離指數衰減，所以同一列裡數值大小差很多。我們寫了一個 ExaGeoStat 式的核心（n = 80000 個隨機位置、分塊 Cholesky、尾端更新為 FP64 矩陣乘法），在 6 個範圍參數上評估對數概似；截斷做在程式內（每個面板在尾端更新前，以依記憶體順序走訪的單次融合 kernel 截一次；早先兩次分開、不合併存取的版本只快 5.2–5.7%）。原生在 1000 W 用到 953–970 W，面板有 88% 的元素比所在列最大值小 2^{-32} 以上——跨度寬、吃滿電，兩者兼具。

上限	截斷 36 位（偏差 2×10^{-5}）	截斷 28 位（7×10^{-4}）	cuBLAS 模擬 36 位（2×10^{-3}）	FP32（87）
1000 W	+2.4／+3.2%，省電 2.6–4.5%	+3.0／+3.9%，省電 7.2–8.7%	慢 41%	慢 8.9／8.2%
700 W	+4.9／+5.1%	+8.2／+7.6%	慢 31%	+3.5／+3.0%
500 W	+5.0／+4.7%	+7.7／+7.4%	慢 21%	+4.3／+3.6%

表中的偏差是對數概似的最大絕對偏差（數值約 10^5）。截斷 28 位在功耗受限時快 7.4–8.2%；1000 W 時這組也快 3.0–3.9% 並省電 7–9%，但 1000 W 的加速隨設定而變（下方依 Morton 排序的 n = 60000 只快 0.0–0.1%，因為原生只有部分時間撞上限）；偏差對參數估計沒有影響。FP32 的偏差大到無法用於估計（n = 8000 時 FP32 仍估出同一個值，n 變大後才偏，可能與條件數隨 n 變差有關）；cuBLAS 模擬既比截斷 28 位不準，又慢 21–41%（尾端更新的 k 等於區塊大小 1024，依第 5.5 節的分界，這個形狀本來就不利於模擬）。另外，這裡的列內跨度比 DFT 更寬，模擬卻沒有算錯，只是精度變差：列內跨度寬是模擬出錯的必要條件，不是充分條件。

最大概似估計（只估範圍參數）。 上表是在固定的 6 個範圍參數上評估；為了確認估出的參數本身不變，我們也跑一維的估計：變異數與平滑度固定，以黃金分割搜尋範圍參數（取對數），每個變體評估 17 次對數概似（n = 60000，1000／700 W，各交錯兩輪；Slurm 工作 3249；gh200/results/gp_mle_summary.csv）。截斷 36 位、28 位與 cuBLAS 模擬估出的範圍參數都與原生完全相同（$\hat{\theta} = 0.05007952$，印出的 8 位全同），最大對數概似只差 4.8×10^{-6} 與 2.3×10^{-4}；FP32 的估計偏了 0.49%，對數概似低 0.72（n = 8000 時連 FP32 都估出同一個值）。700 W 時整個估計以截斷 28 位快 7.1–7.3%、36 位快 4.8–4.9%，耗電少 6.6–6.8% 與 4.5–4.8%；1000 W 時也快 1.8–2.3% 與 1.6–2.2%，耗電少 9.1–9.9% 與 3.5–5.6%；cuBLAS 模擬慢 32–41%。

與 tile 混合精度疊加。 前人的地統計混合精度（第 3.3 節）是把位置依空間曲線排序後，把整個 tile 換成低精度格式。我們在自寫的核心中加入類似的變體：位置依 Morton 曲線排序；尾端更新中，每個 tile 欄在 FP64 範圍以外的 tile（依 tile 範數規則決定，要求精度 10^{-9}，同 [Cao2023]）改用 FP16 輸入、FP32 累加的 tensor core。範圍參數 0.02 時有 52% 的 tile 用 FP16，0.08 時只有 3%。六個範圍參數合計（n = 60000，ABBA 兩輪；Slurm 工作 3244；gh200/results/gp_mp_summary.csv）：

上限	截斷 28 位	混合精度	混合精度 + 截斷 28 位	疊加後再快
1000 W	同速，省電 14.7%（這組原生只有 19–54% 的時間撞上限）	1.62 倍	1.65 倍	1.9%
700 W	+7.4%	1.60 倍	1.71 倍	7.0%
500 W	+7.2%	1.63 倍	1.73 倍	6.1%
對數概似最大偏差	1.3×10^{-4}	0.043	0.043	—

倍數都是相對原生 FP64。截斷可以疊加在換格式之上：混合精度剩下的 FP64 tile 再截到 28 位，功耗受限時再快 6–7%，偏差不變。把 FP16 運算元也截到 5 個尾數位元則沒有額外收益。ExaGeoStatCPP 的混合精度選項（--computation=mixed）只有列舉值、沒有實作，所以無

法與原版直接比較。這組實驗的位置依 Morton 曲線排序，截斷 28 位在 700／500 W 快 7.2–7.4%，與上表（未排序）一致。

真正的 ExaGeoStat。 我們建置了 ExaGeoStatCPP（StarPU 與 Chameleon 的 tile Cholesky，cuBLAS 13.6，tile 大小 1920），以攔截工具截斷它的 FP64 tile GEMM，跑最大概似估計（N = 60000，每次 6 個 BOBYQA 評估；資料以原生產生一次，之後每次都讀同一份；Slurm 工作 3229；gh200/results/exageostat_tdp.csv）。它的 Matérn 共變異數在 CPU 上產生，每次評估約 9 秒，GPU 分解只要 2.3 秒（1000 W）。分解時間的比較（ABBA 兩輪平均）：

上限	截斷 36 位／複製基準	截斷 28 位／複製基準	截斷 28 位／未修改的原生	SM 時脈（複製 → 28 位）
1000 W	快 3.0%	快 4.0%	慢 3.1%	1913 → 1975 MHz
600 W	快 5.0%	快 8.0%	慢 1.3%	1166 → 1305 MHz
450 W	快 5.6%	快 9.4%	慢 0.5%	—

即使在 1000 W，分解也部分受功耗限制（GPU 650–690 W，常有功耗上限降頻事件）。對數概似的相對變化最多 1.2×10^{-10}（36 位）與 9.8×10^{-8}（28 位）。450 W 的兩輪差異大（截斷 28 位對複製基準 +6.9% 與 +11.9%，SM 707 對 926 MHz），因為 CPU 產生共變異數時與 GPU 共用模組上限。攔截工具本身的複製要多花 7–9%，與未修改的 ExaGeoStat 比仍慢 0.5–3.1%；要拿到這個收益，截斷必須寫進函式庫（例如 Chameleon 的 GEMM codelet）。

小結： 在跨度寬又吃滿電的矩陣乘法工作上，整個矩陣用同一種格式的做法（FP32、INT8 模擬）不是結果錯、就是較慢，在測試的三種整個矩陣的做法（FP32、cuBLAS 模擬 36 位、截斷）中，只有截斷又快又準，估出的範圍參數也不變。前人的 tile 混合精度快 1.6 倍，截斷還能疊加在它上面，功耗受限時再快 6–7%。在真正的 ExaGeoStat 上，截斷讓分解比複製基準快 3–9%，但攔截工具的複製成本把它抵銷，需要在函式庫內實作。限制：主要結果來自自寫的核心與合成資料，混合精度變體是簡化的重新實作。

6 討論

6.1 什麼時候值得用

尾數歸零的效益取決於三個條件，少一個，收益就縮小或消失：

1. **系統受功耗限制。** 省下的電只有在功耗上限卡住時脈時才會變成速度。HPL 在 400–800 W 快 4.0–11.0%；調參後的 HPL 在預設的 1000 W 仍有 75% 的時間撞上限，所以也快 3.9%、每次求解少耗電 7.5%。幾乎不受限的程式（例如 1000 W 的 DF-MP2，只有部分時間撞上

限）則主要是省電，速度只多 0.3–0.8%。

2. **被截斷的資料直接、大量地進入乘法器或資料通道。** 矩陣乘法的每個輸入都要乘很多次，所以效益最大；N 體計算第一步就相減，差值的尾數又是滿的；CG 與 SW4 被截的資料只佔記憶體流量的一部分；LAMMPS 的位能表常駐快取，幾乎不經過記憶體。
3. **有機制把誤差攔下來。** HPL 的殘差檢查、HPL-MxP 的迭代修正、SCF 的自洽迭代都會修正或偵測截斷誤差；AI 的權重直接決定輸出品質，截斷就直接轉成精度損失。

表 6.1　各類工作負載在功耗受限時的收益

工作負載	收益	主要原因
有殘差檢查的稠密求解（HPL、HPL-MxP）	HPL 快 4.0–11.0%；HPL-MxP 快 1.8–12.7%（對角優勢測試矩陣）	矩陣乘法主導，殘差檢查把關
高耗電的量子化學（DF-MP2）	快 4.9–8.3%；1000 W 省電 8–14%	矩陣乘法佔 78%、張量建好後截一次即可
跨度寬又吃滿電（高斯過程地統計）	快 7.4–8.2%；1000 W 快 0–4%（依設定）、省電 7–15%	Cholesky 以矩陣乘法為主；FP32 不可用、模擬較慢
自洽迭代（Quantum ESPRESSO）	快 2.4–4.2%（金表面約一半來自迭代變少），總能量印出的每一位都不變	截斷誤差低於當圈容許誤差
高斯基底量子化學（PySCF）	預設格點估計約 2%（400 W 約 4%）；細格點的纈胺黴素在 450–550 W 實測 3.3–3.5%	矩陣乘法只佔總時間 32%
記憶體受限、係數佔流量多（MFEM、SW4）	快 0.3–2.0%	其餘滿尾數的向量稀釋效益
係數常駐快取（LAMMPS）	落在雜訊內	截斷的資料不經記憶體
已用 FP8 的 AI 訓練與推論	訓練變慢；推論以精度 1:1 換速度	FP8 只剩 3 個尾數位元

6.2 與換格式做法的關係

尾數歸零與 INT8 模擬 FP64、FP32／3xTF32、FP8 不是互相取代，而是互補。INT8 模擬在需要的精度較低（約 28 位以下）、或 n 與 k 都達數千以上時較快；截斷在瘦長矩陣、需要 32 位以上精度、以及 GEMMul8 這類耗電的模擬遇到嚴格上限時較快。第 5.5 節的三參數公式在測過的範圍內能事先判斷勝負，可以作為函式庫逐次呼叫選擇的起點（樣本外案例都離分界很遠，還需

要更嚴格的驗證）。FP32 與 3xTF32 的誤差隨內積長度成長，把所有 GEMM 換成它們，在長內積的平面波 DFT 上無法收斂；截斷保留 FP64 累加，誤差與內積長度無關。FP8 已經把 AI 的尾數減到 3 位，在其上再截只是以精度換速度。

整個矩陣換成同一種格式的做法還有一個前提：資料的指數跨度要夠窄。逐張量縮放的格式（FP32、FP8）看整體跨度，逐列切片的 INT8 模擬看同一列內的跨度（第 5.9 節）。DF-MP2 的張量跨度窄，所以內建的 FP32 反而比截斷快；高斯基底 DFT 依運算量加權有 24–44% 的元素（最寬的格點原子軌域矩陣達 87%）比所在列的最大值小 2^{-32} 以上，固定位數的模擬就安靜地算錯了；不過列內跨度寬只是必要條件（高斯過程的跨度更寬卻沒有算錯），而以 tile 為單位選擇格式的混合精度也能處理這類資料。這個跨度可以用攔截工具事先量出來，作為風險指標；cuBLAS 的自動模式也會偵測它而改用原生 FP64，但就不再比截斷快（第 5.9 節）。

所以尾數歸零的定位是：在 FP64 科學計算中，一個每個元素只多一次位元運算、精度 1 位一格可調、誤差不受矩陣形狀與指數跨度影響的預設選項；在跨度窄、矩陣大而方、精度需求較低時，改用模擬或 FP32 可能更快；能依空間排序把小數值集中到整個 tile 的應用（例如地統計），tile 層級的混合精度快得更多，截斷則可以疊加在它剩下的 FP64 tile 上。

6.3 限制與效度威脅

1. **單一平台。** 所有實驗都在同一個 GH200 叢集；其他 GPU 世代的功耗管理不同，效益大小與分界公式的係數可能改變。
2. **保留位數要對多組問題大小驗證。** HPL 的殘差隨矩陣大小跳動，28 位在三組 N 通過（殘差都接近門檻）、一組失敗；本報告以有百倍餘裕的 36 位為操作點。
3. **應用覆蓋有限。** 每個科學軟體只測一到兩個輸入系統，沒有涵蓋幾何最佳化、長時間分子動力學等情境。
4. **攔截工具的複製成本（依應用約 3–9%）。** 所以攔截版一律與「複製但不截斷」的基準比，只有程式內版本與未修改的原生程式比。
5. **節點差異。** 同一功耗上限下，節點間 SM 時脈可差 13–17%，所以所有比較都在同一組節點上成對進行。
6. **散熱條件。** 風扇轉速低時 GPU 先撞溫度上限；風扇全開時原生 HPL-MxP 在 1000 W 會觸發機箱的硬體功耗煞車。這些條件都會改變結論，需一併記錄。
7. **共用叢集。** 其他使用者的工具至少兩次重設叢集的功耗上限；我們在每次執行前檢查實際生效的上限，受影響的量測一律重做。
8. **功耗感測器。** 功耗讀自驅動程式，沒有外接電表；在 A100／H100 上它只取樣部分時間 [Yang2024]。我們的比較都是同一量測路徑下的成對相對值，但絕對能量仍需外接電表驗證。
9. **訓練的收斂點有雜訊。** LoRA 的達標點即使設定完全相同也會差一個評估區間，我們只用了兩個亂數種子，所以只把它當作「FP8 沒有再截的空間」的佐證。

10. **HPL-MxP 的測試矩陣。** 我們用嚴格對角優勢的矩陣與近似的對角分解，規則只允許這種矩陣用於測試 [HPLMxPRules]；「保留 3 位仍穩定」可能只是這個矩陣的性質。
11. **樣本數與節點。** 信賴區間只用於矩陣乘法微基準與 QE；HPL、HPL-MxP、DF-MP2 與高斯過程每格只有兩輪、在一組節點上量。分界點掃描的四個 n 值分別在不同節點上量，跨上限、跨形狀的趨勢可能混入節點差異。
12. **功耗上限的意義。** 1000 W 是預設也是最高上限，不代表不受限：HPL 與 FP64 矩陣乘法在 1000 W 仍有部分時間撞上限。撞上限時 1 秒平均功耗常低於上限，限電器依據的時間尺度不明；低於 450 W 的上限超出規格表的可設範圍 [NVIDIAGH200]。
13. **機制實驗的範圍。** 填值對照只做了保留 0 位與 20 位，500 W 只在一個節點上量兩次；保留 0 位時「數值為 0」的額外省電原因尚未驗證。
14. **收益不全來自功耗。** QE 金表面上，自動調整的收益約一半來自 Davidson 迭代變少（第 5.4 節），原因尚未查明。

6.4 規則合規性

TOP500 的 HPL 要求以完整的 64 位浮點運算求解，明文排除低精度求解再迭代修正、以及用整數運算模擬浮點的方法 [TOP500Linpack]；截斷運算元後的 HPL 也不符合這個要求。HPL-MxP 自 2025 年 6 月起規定最低精度為 16 位浮點，嚴格對角優勢的矩陣只能用於測試 [HPLMxPRules]；保留 3 位的 BF16 實質上只有 12 位，我們的測試矩陣也不符合正式規定。所以本報告所有 HPL／HPL-MxP 數字都是研究結果，不是正式成績。MLPerf Inference 封閉組允許數學上的量化、禁止剪枝，尾數歸零屬於前者 [MLPerfInfRules]；MLPerf Training 的權重能否被截斷要依各項規則判斷，不過就本報告的結果，均勻截斷本來就不划算。科學軟體沒有這類限制，判準是物理結果是否在需要的精度內。

7 結論與未來工作

本專題在現役的 GH200 叢集上，以純軟體把用不到的尾數位元歸零，回答了第 1.4 節的三個問題：

1. **機制：** 省電主要來自低位元不再翻轉（佔 62–97%），數值為 0 在乘法器上再多省一截；這在 tensor core、一般 FP64 運算單元與記憶體搬移上都成立。
2. **效益：** 功耗受限、截斷做在程式內時，以矩陣乘法為主、有正確性檢查的完整程式都得到正面收益：HPL（調參後）保留 36 位快 4.0–11.0%（等效多出 39–127 W），HPL-MxP 保留 3 位快 1.8–12.7%（對角優勢的測試矩陣），Quantum ESPRESSO 依收斂門檻自動調整位元數快 2.4–4.2%、少耗電 2.5–3.9%，總能量印出的每一位都不變；高耗電的 DF-MP2 快 4.9–8.3%，跨度寬又吃滿電的高斯過程地統計快 7.4–8.2%。在預設的 1000 W 上限下，HPL 也快 3.9%、每次求解少耗電 7.5%，DF-MP2 少 8–14%。代價是必須對多組問題大小驗證保留位數，並使用最近捨入。

3. **適用範圍：** 它與 INT8 模擬 FP64 互補，分界點可以用三參數公式事先估計（樣本外 8／8 判對，但案例都離分界很遠）；把所有 GEMM 換成 FP32／3xTF32 在長內積上不能取代它；列內指數跨度寬時，整個矩陣換格式可能出錯（逐張量格式放不下、固定位數的逐列切片模擬可能安靜地算錯），截斷則不受影響，也能疊加在 tile 層級的混合精度上；在已用 FP8 的 AI 上，FP8 已拿走幾乎全部的好處。非矩陣乘法的程式也省電，但在真實軟體中只剩 0.3–2.0%。

一句話總結：在功耗受限的 FP64 科學計算中，尾數歸零是一個每個元素只多一次位元運算、不需要新硬體、精度 1 位一格可調的調整手段，最適合有殘差修正或自洽迭代的求解器。

未來工作：

- **第二種硬體：** 在 H200、MI300A 或 Blackwell 上重做分界點掃描與 HPL，檢驗結論與公式能否移植；HPL-MxP 也要改用規則允許的矩陣產生器與精確的對角分解，重做保留位數掃描。
- **函式庫整合：** 把「依形狀與上限選擇截斷或模擬」的規則做進矩陣乘法的函式庫（例如 cuBLAS 或 Chameleon 的 GEMM codelet），讓截斷不需要額外複製；ExaGeoStat 上複製成本剛好抵銷了收益。
- **更多自適應求解器：** 把「依容許誤差決定位元數」推廣到其他迭代法，例如 Krylov 求解器的內層與混合精度迭代修正；也可以用資料的位元資訊量事先估計需要保留的位數 [Klower2021]，減少試驗。
- **端到端比較：** 在 GPU4PySCF 中實作依形狀選擇，與 Huang 等人的自適應 INT8 模擬直接比較 [Huang2026]。
- **外接電表：** 以外接電表交叉驗證絕對能量。

參考文獻

- [AMD2022] P. V. Argade, D. N. Peroni (Advanced Micro Devices, Inc.). Variable precision computing system. US Patent 11,455,766 B2, 2022. https://patents.google.com/patent/US11455766B2/en
- [Antepara2025] O. Antepara, Z. Zhao, B. Austin, N. Ding, L. Oliker, N. J. Wright, S. Williams. Benchmark-driven Models for Energy Analysis and Attribution of GPU-Accelerated Supercomputing. SC '25, pp. 888–904. doi:10.1145/3712285.3759815
- [Bhalachandra2022] S. Bhalachandra, B. Austin, S. Williams, N. J. Wright. Understanding the Impact of Input Entropy on FPU, CPU, and GPU Power. arXiv:2212.08805, 2022.
- [Chatterjee2017] N. Chatterjee, M. O'Connor, D. Lee, D. R. Johnson, S. W. Keckler, M. Rhu, W. J. Dally. Architecting an Energy-Efficient DRAM System for GPUs. HPCA 2017, pp. 73–84. doi:10.1109/HPCA.2017.58

- [Dawson2017] A. Dawson, P. D. Düben. rpe v5: an emulator for reduced floating-point precision in large numerical simulations. Geosci. Model Dev. 10:2221–2230, 2017. doi:10.5194/gmd-10-2221-2017
- [Dawson2024] W. Dawson, K. Ozaki, J. Domke, T. Nakajima. Reducing Numerical Precision Requirements in Quantum Chemistry Calculations. J. Chem. Theory Comput. 20(24):10826–10837, 2024. doi:10.1021/acs.jctc.4c00938
- [Dongarra2025] J. Dongarra, P. Luszczek. HPL-MxP Benchmark: Mixed-Precision Algorithms, Iterative Refinement, and Scalable Data Generation. Int. J. High Perform. Comput. Appl. 40(1):52–62, 2026（2025 年 9 月線上發表）. doi:10.1177/10943420251382476；arXiv:2509.19618
- [Fasi2021] M. Fasi, N. J. Higham, M. Mikaitis, S. Pranesh. Numerical behavior of NVIDIA tensor cores. PeerJ Computer Science 7:e330, 2021. doi:10.7717/peerj-cs.330
- [Ghose2018] S. Ghose, A. G. Yağlıkçı, R. Gupta, D. Lee, K. Kudrolli, W. X. Liu, H. Hassan, K. K. Chang, N. Chatterjee, A. Agrawal, M. O'Connor, O. Mutlu. What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study. Proc. ACM Meas. Anal. Comput. Syst. 2(3), 2018. doi:10.1145/3224419
- [Giannozzi2020] P. Giannozzi et al. Quantum ESPRESSO toward the exascale. J. Chem. Phys. 152, 154105, 2020.
- [Gregersen2024] T. Gregersen, P. Patel, E. Choukse. Input-Dependent Power Usage in GPUs. SC24-W, pp. 1872–1877, 2024. doi:10.1109/SCW63240.2024.00235
- [Guenther2016] D. Guenther, T. Henriksson, R. Leupers, G. Ascheid. Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing. DATE 2016, pp. 1028–1029.
- [He2024] H. He. Strangely, Matrix Multiplications on GPUs Run Faster When Given "Predictable" Data! [short]. Thonk From First Principles (blog), April 29, 2024. https://www.thonking.ai/p/strangely-matrix-multiplications
- [Higham2019] N. J. Higham, S. Pranesh. Simulating Low Precision Floating-Point Arithmetic. SIAM J. Sci. Comput. 41(5):C585–C602, 2019. doi:10.1137/19M1251308
- [Huang2026] H. Huang, W. Shao, J. Hammond. Accelerating Density Fitting with Adaptive-precision and 8-bit Integer on AI Accelerators. arXiv:2601.08077, 2026.
- [Lucas2016] J. Lucas, B. Juurlink. ALUPower: Data Dependent Power Consumption in GPUs. MASCOTS 2016, pp. 95–104. doi:10.1109/MASCOTS.2016.21
- [MLPerfInfRules] MLCommons. MLPerf Inference Rules, section 8.1 Weight Definition and Quantization. https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc

- [OConnor2017] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, W. J. Dally. Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems. MICRO 2017, pp. 41–54. doi:10.1145/3123939.3124545
- [Ootomo2024] H. Ootomo, K. Ozaki, R. Yokota. DGEMM on Integer Matrix Multiplication Unit. Int. J. High Perform. Comput. Appl. 38(4):297–313, 2024. doi:10.1177/10943420241239588
- [Ozaki2025] K. Ozaki, Y. Uchino, T. Imamura. Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique. arXiv:2504.08009, 2025.
- [Pool2008] J. Pool, A. Lastra, M. Singh. Energy-precision tradeoffs in mobile graphics processing units. ICCD 2008, pp. 60–67. doi:10.1109/ICCD.2008.4751841
- [Pool2011] J. Pool, A. Lastra, M. Singh. Precision selection for energy-efficient pixel shaders. HPG 2011, pp. 159–168. doi:10.1145/2018323.2018349
- [RubioGonzalez2013] C. Rubio-González, C. Nguyen, H. D. Nguyen, J. Demmel, W. Kahan, K. Sen, D. H. Bailey, C. Iancu, D. Hough. Precimonious: Tuning Assistant for Floating-Point Precision. SC13. doi:10.1145/2503210.2503296
- [Tong2000] J. Y. F. Tong, D. Nagle, R. A. Rutenbar. Reducing Power by Optimizing the Necessary Precision/Range of Floating-Point Arithmetic. IEEE TVLSI 8(3):273–286, 2000. doi:10.1109/92.845894
- [Uchino2025] Y. Uchino et al. High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines. SC '25 Workshops. doi:10.1145/3731599.3767539
- [Woo2023] J. Woo, S. Kim, W. Y. Kim. Dynamic Precision Approach for Accelerating Large-Scale Eigenvalue Solvers in Electronic Structure Calculations on Graphics Processing Units. J. Chem. Theory Comput. 19(5):1457–1465, 2023. doi:10.1021/acs.jctc.2c00983
- [Wu2025] X. Wu, Q. Sun, et al. Enhancing GPU-Acceleration in the Python-Based Simulations of Chemistry Frameworks. WIREs Comput. Mol. Sci. 15(2):e70008, 2025. doi:10.1002/wcms.70008（預印本 arXiv:2404.09452）
- [Li2025] R. Li, Q. Sun, X. Zhang, G. K.-L. Chan. Introducing GPU Acceleration into the Python-Based Simulations of Chemistry Framework. J. Phys. Chem. A 129(5):1459–1468, 2025. doi:10.1021/acs.jpca.4c05876
- [OlivaresAmaya2010] R. Olivares-Amaya, M. A. Watson, R. G. Edgar, L. Vogt, Y. Shao, A. Aspuru-Guzik. Accelerating Correlated Quantum Chemistry Calculations Using Graphical Processing Units and a Mixed Precision Matrix Multiplication Library. J. Chem. Theory Comput. 6(1):135–144, 2010. doi:10.1021/ct900543q

- [Abdulah2018] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, D. E. Keyes. ExaGeoStat: A High Performance Unified Software for Geostatistics on Manycore Systems. IEEE TPDS 29(12):2771–2784, 2018. doi:10.1109/TPDS.2018.2850749
- [Abdulah2019] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, D. E. Keyes. Geostatistical Modeling and Prediction Using Mixed Precision Tile Cholesky Factorization. HiPC 2019, pp. 152–162. doi:10.1109/HiPC.2019.00028
- [Abdulah2022] S. Abdulah, Q. Cao, Y. Pei, G. Bosilca, J. Dongarra, M. G. Genton, D. E. Keyes, H. Ltaief, Y. Sun. Accelerating Geostatistical Modeling and Prediction With Mixed-Precision Computations: A High-Productivity Approach With PaRSEC. IEEE TPDS 33(4):964–976, 2022. doi:10.1109/TPDS.2021.3084071
- [Cao2022] Q. Cao et al. Reshaping Geostatistical Modeling and Prediction for Extreme-Scale Environmental Applications. SC22, pp. 1–12. doi:10.1109/SC41404.2022.00007
- [Cao2023] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, G. Bosilca. Reducing Data Motion and Energy Consumption of Geospatial Modeling Applications Using Automated Precision Conversion. IEEE CLUSTER 2023, pp. 330–342. doi:10.1109/CLUSTER52292.2023.00035
- [Abdulah2024] S. Abdulah et al. Boosting Earth System Model Outputs And Saving PetaBytes in Their Storage Using Exascale Climate Emulators. SC24, pp. 1–12. doi:10.1109/SC41406.2024.00008
- [Ren2025] J. Ren, H. Ltaief, S. Abdulah, D. E. Keyes. Accelerating Mixed-Precision Out-of-Core Cholesky Factorization with Static Task Scheduling. ISC High Performance 2025, pp. 1–12. doi:10.23919/ISC.2025.11017733
- [Grutzmacher2020] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, E. S. Quintana-Ortí. Acceleration of PageRank with Customized Precision Based on Mantissa Segmentation. ACM TOPC 7(1):1–19, 2020. doi:10.1145/3380934
- [cuBLASEmu] NVIDIA. cuBLAS Library Documentation, §1.5 Floating Point Emulation（v13.4；另核對 13.2.1 存檔版）. Accessed Oct. 5, 2026. https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation
- [Schwarz2026] A. Schwarz et al. Guaranteed DGEMM Accuracy While Using Reduced Precision Tensor Cores Through Extensions of the Ozaki Scheme. SCA/HPCAsia 2026, pp. 91–101. doi:10.1145/3773656.3773670
- [Yang2024] Z. Yang, K. Adamek, W. Armour. Accurate and Convenient Energy Measurements for GPUs: A Detailed Study of NVIDIA GPU's Built-In Power Sensor. SC24. doi:10.1109/SC41406.2024.00028

- [Ozaki2012] K. Ozaki, T. Ogita, S. Oishi, S. M. Rump. Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications. Numerical Algorithms 59(1):95–118, 2012. doi:10.1007/s11075-011-9478-1
- [Rountree2012] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, M. Schulz. Beyond DVFS: A First Look at Performance under a Hardware-Enforced Power Bound. IPDPSW 2012, pp. 947–953. doi:10.1109/IPDPSW.2012.116
- [Patki2013] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, B. R. de Supinski. Exploring Hardware Overprovisioning in Power-Constrained, High Performance Computing. ICS '13, pp. 173–182. doi:10.1145/2464996.2465009
- [Inadomi2015] Y. Inadomi et al. Analyzing and Mitigating the Impact of Manufacturing Variability in Power-Constrained Supercomputing. SC '15, Article 78. doi:10.1145/2807591.2807638
- [Zeus2023] J. You, J.-W. Chung, M. Chowdhury. Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training. NSDI '23, pp. 119–139. https://www.usenix.org/conference/nsdi23/presentation/you
- [Malossi2018] A. C. I. Malossi et al. The Transprecision Computing Paradigm: Concept, Design, and Applications. DATE 2018, pp. 1105–1110. doi:10.23919/DATE.2018.8342176
- [Klower2021] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, T. N. Palmer. Compressing atmospheric data into its real information content. Nature Computational Science 1(11):713–724, 2021. doi:10.1038/s43588-021-00156-2
- [NVIDIAGH200] NVIDIA. NVIDIA GH200 Grace Hopper Superchip Datasheet, March 2025. https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/ (accessed Oct. 5, 2026)
- [TOP500Linpack] TOP500. The Linpack Benchmark. https://www.top500.org/project/linpack/ (accessed Oct. 5, 2026)
- [HPLMxPRules] HPL-MxP. Rules. https://hpl-mxp.org/rules.md (accessed Oct. 5, 2026)

附錄：實驗環境、程式位置與重現方式

A.1 環境

項目	內容
硬體	14 個節點，各一顆 NVIDIA GH200 480GB（aarch64），200 Gb/s InfiniBand，Slurm 分區 compute

項目	內容
編譯與函式庫	NVHPC 26.5（CUDA 13.2、cuBLAS 13、NCCL、HPC-X Open MPI）；ExaGeoStatCPP 連結 cuBLAS 13.6
基準	HPL（rocHPL 的 CUDA 移植，feat/cuda-port）、HPL-MxP（perf/h200-mantissa）[Dongarra2025]
科學軟體	Quantum ESPRESSO 7.6 [Giannozzi2020]、GPU4PySCF 1.8.1 [Wu2025, Li2025]、MFEM（提交 c362dea）、SW4（提交 52b51c9）、LAMMPS stable_22Jul2025_update6、GEMMul8 3.5.1
AI	MLPerf Training Llama 2 70B LoRA，NVIDIA v5.0 實作（8 節點）
分析	Python（pandas、numpy），虛擬環境 gh200/.venv

A.2 程式位置

路徑	用途
gh200/bench/	cuBLAS 尾數矩陣乘法微基準（第 5.2、5.5 節）
gh200/shim/mantshim.cu	cuBLAS 攔截函式庫：截斷（MANT_BITS、MANT_ROUND）、cuBLAS 模擬、單精度替換（MANT_F32）、cuBLAS 自動模式與位數紀錄（MANT_EMU_DYN、MANT_EMU_LOG）、運算元統計與傾印
apps/q-e/KS_Solvers/Davidson/	Quantum ESPRESSO 的程式內截斷與自動調整規則（QE_MANT_ADAPT）
rocHPL/src/blas/HPL_quantize_device.cpp	HPL 的運算元捨入（HPL_QUANT_A_BITS 等）
rocHPL-MxP/src_mi350/hplmxp.h	HPL-MxP 寫回 BF16 時的歸零（編譯時 MANT_BITS）
apps/（見 apps/建置日誌.md）	MFEM、SW4、LAMMPS 的係數截斷插入點與建置方式
apps/pyscf-inputs/mp2_variants.py	DF-MP2 各變體：程式內截斷 DF 張量、內建 FP32、cuBLAS 模擬（第 5.8 節）

路徑	用途
apps/geostat/gp_mle.py	高斯過程對數概似與最大概似估計：截斷、cuBLAS 模擬、FP32、Morton 排序與 tile 混合精度（第 5.10 節）
apps/exageostatcpp/、apps/patches/	ExaGeoStatCPP（submodule）與在 GH200、CUDA 13 上建置所需的修正（建置日誌 §9）
paper/	英文 LaTeX 論文稿（main.tex、refs.bib、figs/、main.pdf）
gh200/refs/	各輪文獻查證紀錄（round4_refs.md、round5_refs.md）
gh200/slurm/	工作腳本：e1_gemm_tdp.sbatch（微基準與形狀掃描）、hpl_tdp.sbatch（HPL／HPL-MxP）、sci_sweep.sbatch（科學軟體）、app_tdp.sbatch（第三方軟體）、mp2_tdp.sbatch（DF-MP2）、gp_tdp.sbatch（高斯過程）、exageostat_tdp.sbatch（ExaGeoStat）、hpl_tune.sbatch（HPL 調參）、hpl_shim.sbatch（單 GPU HPL 經攔截工具：逐列跨度與模擬）
gh200/analysis/	summarize.py、sci_summary.py、app_summary.py、crossover.py、qe_adapt_ci.py、range_sweep.py、span_report.py、mp2_summary.py、gp_summary.py、exa_summary.py、paper_figures.py（英文論文的向量圖）、report_static_figs.py（報告圖 5.9–5.13）；圖 5 與圖 5.13 的座標來自 results/range_map_points.csv
gh200/results/	所有彙整後的 CSV，圖說裡的檔名都在這裡
gh200/實驗日誌.md、gh200/應用實驗日誌.md	完整的實驗過程紀錄（含失敗與重做）

A.3 重現方式

所有工作都以 Slurm 送出，腳本只在自己獨佔的節點上設定功耗上限，結束時恢復 1000 W，每次執行前檢查實際生效的上限。例如：

```
module load nvhpc/26.5
cd gh200
# 分界點掃描（第 5.5 節）
sbatch -w work1 --export=ALL,TDPS="1000 575 450",SHAPES="8192x4096x16384
8192x256x81920",CONFIGS="fp64:52 fp64:36 fp64emu:32
gemmul8:10",WARMUP=2,SECONDS_PER=6 slurm/e1_gemm_tdp.sbatch
.venv/bin/python analysis/crossover.py <工作編號 ...>
# Quantum ESPRESSO 自動調整位元（第 5.4 節）
sbatch -w work2 --
export=ALL,APP=qe,CASE=si1000,INP=si1000.in,TDPS=550,VARIANTS="native
adapt0:nearest src28:nearest src28:nearest adapt0:nearest native"
slurm/sci_sweep.sbatch
.venv/bin/python analysis/sci_summary.py logs/sci/<工作名>_<編號>
.venv/bin/python analysis/qe_adapt_ci.py <彙整 CSV> src28:nearest
# 第三方軟體（第 5.6 節）
sbatch -w work10 --export=ALL,APP=mfem slurm/app_tdp.sbatch
.venv/bin/python analysis/app_summary.py logs/apps/<工作名>_<編號>
```

每張圖的圖說都列出對應的 Slurm 工作編號與結果檔，可以從 gh200/logs/ 與 gh200/results/ 找到原始記錄。